

Cross-Chain Content Rights Management Service

A Polkadot-Based Unified Rights Token for Subscription, Pay-Per-View, and Permanent Ownership

Ryc Brownrigg

Supervised by Dr Joshua Ellul

Centre for Distributed Ledger Technologies
Faculty of ICT
University of Malta

May, 2026

A dissertation submitted in partial fulfilment of the requirements for the degree of M.Sc. in Blockchain and Distributed Ledger Technologies (ICT).



Copyright ©2026 University of Malta

WWW.UM.EDU.MT

First edition, June 25, 2026



**L-Università
ta' Malta**

Declaration by Postgraduate Students

(a) Authenticity of Dissertation

I hereby declare that I am the legitimate author of this Dissertation and that it is my original work.

No portion of this work has been submitted in support of an application for another degree or qualification of this or any other university or institution of higher education. I hold the University of Malta harmless against any third party claims with regard to copyright violation, breach of confidentiality, defamation and any other third party right infringement.

(b) Research Code of Practice and Ethics Review Procedures

I declare that I have abided by the University's Research Ethics Review Procedures.

As a Master's student, as per Regulation 58 of the General Regulations for University Postgraduate Awards, I accept that should my dissertation be awarded a Grade A, it will be made publicly available on the University of Malta Institutional Repository.

Faculty/Institute/Centre/School	Faculty of ICT
Degree	M.Sc. in Blockchain and Distributed Ledger Technologies (ICT)
Title	Cross-Chain Content Rights Management Service
Candidate (Id.)	Ryc Brownrigg (2104001)

Signature of Student

Date

June 25, 2026

Acknowledgements

I express my deepest gratitude to my supervisor, Prof. Joshua Ellul, for his insightful guidance, patience, and unwavering support throughout this dissertation.

I am also grateful to the faculty of the Centre for Distributed Ledger Technologies for creating the academic environment in which this work was produced.

To the Polkadot and Kusama communities, and especially to Parity Technologies for the Canary Chaos program, which provided free, dedicated parachain slots and enabled real-world chaos testing at zero cost. Without this opportunity, the prototype would not have been feasible within the timeline.

I am particularly grateful to Grammarly, which served as an indispensable tool for grammar, spelling, clarity, and rephrasing throughout the writing and revision process. I am also grateful to SuperWhisper, the dictation software that allowed me to compose most of this dissertation by voice rather than typing, making the writing process far more accessible and efficient given my limited typing speed.

Finally, I want to thank my mother, whose endless love, encouragement, and faith in me have carried me through all my wild and wonderful adventures. I love you so much, Mom.

Thank you all.

Abstract

The content creator economy concentrates value in centralized intermediaries; no existing system delivers subscription, pay-per-view, and permanent-ownership access rights through a single cross-chain-portable primitive. This dissertation asks: *how can a Polkadot parachain built on FRAME pallet primitives and XCM deliver a unified rights token natively supporting all three monetization models across heterogeneous blockchain networks, and what levels of transaction finality, cost efficiency, and creator revenue retention can such a system achieve?*

CCRMS (Cross-Chain Content Rights Management Service) is a Polkadot parachain implementing parent-child NFT nesting on pallet-nfts, automatic royalty splits, and cross-chain portability via XCM and Snowbridge. The prototype is evaluated with 56 unit tests, seven performance benchmarks, a security audit, a centralized Express.js/SQLite baseline comparison, and a 10,000-iteration Monte Carlo creator revenue simulation.

All 11 KPIs met their targets: 26.2 TPS sustained throughput, ~6-second inclusion latency, 3–5-block cross-chain finality, 191 bytes per content item, 90% baseline uptime, and 15–20-second MTTR. The Monte Carlo simulation yields ~16.8% higher average creator revenue than centralized platforms, but this advantage is **strongly audience-dependent**: creators above ~10,000 followers benefit decisively, while those below ~2,000 are better served by centralized discovery platforms.

Three findings advance the state of knowledge. First, a unified rights token supporting all three access models can be constructed from standard FRAME primitives without novel cryptographic mechanisms, closing literature gap G7. Second, the revenue advantage of decentralized rights management is not universal, crossing a follower threshold at approximately 2,000. Third, the work demonstrates that conservatively chosen architectural decisions—specifically the pallet-first design—provide incidental resilience against ecosystem-level changes, as evidenced by the January 2026 discontinuation of ink! development.

Keywords: cross-chain content rights, Polkadot XCM, unified rights token, creator revenue retention, design science research

Contents

List of Abbreviations	xix
1 Introduction	1
1.1 The Problem	1
1.2 Background Context	2
1.3 Research Aim and Approach	3
1.4 Contributions	4
1.5 Scope and Limitations	4
2 Background and Related Work	7
2.1 Blockchain-Based Content Management	7
2.1.1 Market Context	7
2.1.2 Foundational Work on Blockchain-Based DRM	8
2.1.3 NFTs and Intellectual Property Rights	8
2.2 Cross-Chain Interoperability	9
2.2.1 Polkadot Architecture	9
2.2.2 Cross-Chain Technology Surveys	10
2.2.3 Bridges and Snowbridge	10
2.2.4 Polkadot 2.0 and JAM	11
2.2.5 Cross-Chain Copyright Management as a Research Target	11
2.3 Decentralized Monetization Models	11
2.3.1 Creator Economy Context	11
2.3.2 DeFi and Web3 Foundations	12
2.3.3 Subscription, Pay-Per-View, and Purchase Models	12
2.3.4 Pay-Per-View and the Micropayment Problem	12

2.3.5	Smart Contracts as the Implementation Layer	13
2.3.6	Royalty Automation	13
2.4	Challenges and Gaps	13
2.5	Summary	17
3	Research Questions and Objectives	19
3.1	Primary Research Question	19
3.2	Sub-Questions	20
3.3	Research Objectives	21
3.4	Mapping Questions to Objectives	22
3.5	Summary	23
4	Methodology and System Design	25
4.1	Research Methodology	25
4.1.1	Design Science Research Framing	25
4.1.2	Iterative Prototyping	26
4.1.3	Data Collection Methods	26
4.1.4	Evaluation Philosophy: Measurement-Focused	26
4.1.5	Methodology Pivots	27
4.2	System Design	27
4.2.1	The Unified Rights Token Concept	27
4.2.2	The Three Monetization Models	28
4.2.3	NFT-Based Rights Tokenization	29
4.2.4	Cross-Chain Communication Model	30
4.2.5	Pallet-First Architecture	31
4.2.6	Layered Architecture	32
5	Implementation	35
5.1	Development Environment	35
5.1.1	Toolchain and Language	35
5.1.2	Workspace Layout	36
5.1.3	Local Development Network	36
5.1.4	Test Infrastructure	36
5.2	Pallet Implementation and Smart Contract Layer	37
5.2.1	The pallet-content-rights Design	37
5.2.2	NFT Nesting via pallet-nfts	38
5.2.3	Royalty Propagation	39
5.2.4	The ink! Contract Wrapper and EVM Precompile	39

5.3	Cross-Chain Integration	40
5.3.1	XCM Message Flows	40
5.3.2	Snowbridge Integration with Ethereum	41
5.4	Challenges and Solutions	42
5.5	Summary	44
6	Evaluation and Testing	45
6.1	Evaluation Approach	45
6.2	Testing Layers	46
6.3	Performance Testing Framework	48
6.4	Test Scenarios	49
6.5	Metrics and Key Performance Indicators	50
6.6	Testing Infrastructure	52
6.7	Validity and Limitations	53
6.8	Summary	54
7	Results	55
7.1	Technical Results	55
7.1.1	Throughput	55
7.1.2	Latency	56
7.1.3	Block Weight Utilization	57
7.1.4	Storage Growth	57
7.1.5	Reliability	58
7.1.6	Snowbridge End-to-End Bridge	59
7.1.7	Comparative Analysis vs. Centralized Baseline	60
7.2	Economic Results	60
7.2.1	Monte Carlo Creator Revenue Simulation	61
7.2.2	Audience Size Dependency	62
7.2.3	Comparative Fee Analysis	64
7.3	Security and User-Centric Results	65
7.3.1	Security Audit Findings	65
7.3.2	Test Coverage	66
7.3.3	Decentralization Assessment (HHI)	66
7.3.4	Static Analysis and Dependency Audit	67
7.4	Summary	67
8	Discussion	69
8.1	Answering the Research Questions	69

8.1.1	SQ1: Cross-Chain Rights Operations and Finality	69
8.1.2	SQ2: Latency versus Decentralization	69
8.1.3	SQ3: Unified Rights Token	70
8.1.4	SQ4: Security and Authorization	70
8.1.5	SQ5: Economic Viability	70
8.2	Interpretation of Key Results	71
8.2.1	The 26.2 TPS Figure in Context	71
8.2.2	The Economic Crossover at ~2,000 Followers	71
8.2.3	The Snowbridge End-to-End Demonstration	72
8.3	Architectural Decisions Revisited	72
8.4	The Work in Context	72
8.5	Limitations	73
8.6	Implications	74
8.7	Critical Self-Reflection	75
8.8	Summary	75
9	Conclusion and Future Work	77
9.1	Summary of Contributions	77
9.2	Principal Findings	78
9.3	Future Work	78
9.4	Closing Reflection	80
	Appendix A Implementation Challenge Narratives	81
A.1	The Pivot from RMRK 2.0 to pallet-nfts	81
A.2	Contract-First to Pallet-First Pivot	82
A.3	The Snowbridge Fork-Version Configuration Issue	82
A.4	The Lodestar Preset Version Pinning	83
A.5	The XCM Sovereign Account Endianness Bug	83
A.6	The XCM Fee Scaling for Proof Size	84
A.7	Finding B: Authorization Gap	84
A.8	Auto-Renewal Child Cap	84
A.9	ink! Discontinuation	85
	Appendix B Research Objective Details	87
B.1	RO1: Design and Prototype XCM Extensions	87
B.2	RO2: Implement a Unified Rights Token	87
B.3	RO3: Merge Subscription, PPV, and Purchase Logic	87
B.4	RO4: Benchmark Scalability and Interoperability	88

B.5	RO5: Zero-Knowledge Proof Hooks (Not Achieved)	88
B.6	RO6: Quantify Creator Savings	88
Appendix C Code Listings		89
C.1	Solidity Interface: ContentRightsPrecompile	89
C.2	Canonical XCM Message	90
C.3	Repository Directory Layout	90
C.4	Dispatchable Extrinsic Reference	91
C.5	Pallet Storage Layout Reference	92
Appendix D Security and Coverage Findings		95
D.1	Finding B: Authorization Gap	95
D.2	Security Audit Finding E: View Pack Overwrite	95
D.3	Accepted Design Findings (A, C, D)	95
D.4	Error Variant Coverage	96
D.5	Static Analysis Advisory Breakdown	96
Appendix E Future Work Details		99
E.1	Production Deployment	99
E.2	Migration to Polkadot 2.0 Features	99
E.3	Migration Paths for the Smart Contract Layer	99
E.4	The 50-Child Cap	100
E.5	User Study and Consumer Application	100
E.6	The Bridged-Payment Final Hop	100
E.7	Resale Royalties	100
E.8	Capability-Based Delegation	101
E.9	Monte Carlo Simulation Extensions	101
E.10	Formal Verification	101
E.11	JAM Transition	101
Appendix F Work Plan and Timeline		103
References		105

List of Tables

2.1	Literature Gaps and CCRMS Contributions	14
3.1	Research Sub-Questions	20
3.2	Research Objectives Summary	21
3.3	Mapping of Sub-Questions to Objectives, Methods, and Chapters	22
5.1	Implementation Challenges Summary	42
6.1	Mapping of Evaluation Activities to Research Sub-Questions	45
6.2	Benchmark Scripts	48
6.3	Test Scenarios	49
6.4	KPI Definitions	51
6.5	KPI Results Summary	51
6.6	Test Environment Specifications	52
7.1	Single-Chain Throughput by Extrinsic Type and Batch Size	56
7.2	XCM Cross-Chain Latency by Operation	57
7.3	Block Weight Utilization at Increasing Batch Sizes	57
7.4	Per-Item Storage Costs (SCALE-encoded)	58
7.5	Reliability Test Results	58
7.6	Snowbridge End-to-End Message Path (Verified)	59
7.7	Comparative Analysis: CCRMS Parachain vs. Centralized (Express.js + SQLite)	60
7.8	Monte Carlo Creator Revenue Simulation (Mean Annual Revenue Per Creator)	61
7.9	CCRMS Advantage by Creator Audience Tier	62
7.10	Per-Scenario Break-Even Rates	63
7.11	95% Confidence Intervals for Total Simulated Revenue (1,000 Creators)	63

7.12 Assumed Fee Ranges for Each Platform Category	64
7.13 Security Audit Findings	65
7.14 Herfindahl-Hirschman Index (HHI) Comparison	66
9.1 Future Work Summary	79
C.1 Dispatchable Extrinsic in pallet-content-rights	91
C.2 Storage Layout of pallet-content-rights	92
D.1 Error Variant Coverage	96
D.2 Static Analysis Advisory Breakdown	96

List of Figures

4.1	The three monetization models natively supported by the unified rights token (addresses SQ3). A single content registration may offer all three models concurrently; <code>check_access</code> resolves in priority order: Ownership > Subscription > PPV.	29
4.2	Parent-child NFT nesting model. Each content registration creates one NFT collection containing a parent Content NFT (the canonical on-chain identity of the content) and zero or more child rights NFTs, each owned by an individual user and tagged with their access mode.	30
4.3	XCM cross-chain rights operation (addresses SQ1 and SQ4). The three-instruction sequence dispatches an <code>xcm_*</code> extrinsic on the CCRMS parachain, decoupling the paying account (Para B sovereign account) from the rights beneficiary (the end user). The same pattern applies to all five <code>xcm_*</code> extrinsics.	31
4.4	CCRMS six-layer architecture (addresses SQ1 and SQ2). Each layer is independently replaceable: Layer 5 (smart contracts) may be removed, Layer 6 (XCM) may upgrade from v3 to v5, and Layer 1 (relay chain) may migrate to JAM, without modifying the business logic in Layer 3.	33
5.1	CCRMS implementation-level component diagram. Solid arrows indicate active extrinsic dispatch; dashed arrows indicate read-only storage access. <code>pallet-content-rights</code> is the sole authoritative state layer; the <code>ink!</code> contract and EVM precompile are stateless wrappers that forward calls or read storage respectively. XCM delivers cross-chain calls via HRMP. <code>pallet-rights-verifier</code> enables remote parachains to validate rights state via Merkle storage proofs without oracles or relayers.	37

- 6.1 Four-layer testing architecture (Layer 1 at base: fastest and most granular, run on every commit; Layer 4 at apex: broadest scope, run infrequently). Each layer tests what lower layers cannot: unit tests isolate individual extrinsic logic; the XCM simulator isolates message-construction failures; Zombienet provides definitive wall-clock performance evidence; Snowbridge validates the complete Ethereum bridge pathway. 47
- 7.1 Monte Carlo simulation structure (1,000 creators $\times$ 10,000 iterations = 10 million revenue paths per platform). Creator parameters are sampled per creator; the platform fee is drawn uniformly from each platform's range at each iteration. The four paths run in parallel and are aggregated independently, yielding the mean revenue figures in Table 7.8, the 95% confidence intervals in Table 7.11, and the audience-size crossover finding in Table 7.9. 61

List of Abbreviations

ABI	Application Binary Interface
AI	Artificial Intelligence
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BLS	Boneh–Lynn–Shacham (signature scheme)
CAGR	Compound Annual Growth Rate
CCRMS	Cross-Chain Content Rights Management Service
CLI	Command-Line Interface
CPU	Central Processing Unit
DEX	Decentralized Exchange
DOT	Polkadot native token
DRM	Digital Rights Management
DSR	Design Science Research
ESMA	European Securities and Markets Authority
ETH	Ether (Ethereum’s native token)
EU	European Union
EVM	Ethereum Virtual Machine
FRAME	Framework for Runtime Aggregation of Modularized Entities
GB	Gigabyte
H160	160-bit hash (Ethereum-style address)
HHI	Herfindahl-Hirschman Index
HRMP	Horizontal Relay-routed Message Passing
HTTP	Hypertext Transfer Protocol
ID	Identifier

IPFS	InterPlanetary File System
JAM	Join-Accumulate Machine
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MEV	Maximal Extractable Value
MiCA	Markets in Crypto-Assets (EU Regulation)
MTTR	Mean Time To Recovery
NFT	Non-Fungible Token
OTT	Over-The-Top (media service)
PIL	Programmable IP License (Story Protocol)
PolkaVM	Polkadot Virtual Machine
PPV	Pay-Per-View
RISC	Reduced Instruction Set Computer
RMRK	Remark (NFT protocol; derived from the <code>system.remark</code> extrinsic)
RPC	Remote Procedure Call
SCALE	Simple Concatenated Aggregate Little-Endian (Substrate codec)
SDK	Software Development Kit
SIGTERM	Signal Terminate (POSIX signal)
SQL	Structured Query Language
TLS	Transport Layer Security
TPS	Transactions Per Second
TVL	Total Value Locked
UI	User Interface
USD	United States Dollar
WASM	WebAssembly
XCM	Cross-Consensus Messaging
ZK	Zero-Knowledge (proof / cryptography)

Introduction

1.1 | The Problem

The prevailing content economy is unfavorable to creators. An independent musician earning USD 100,000 in streaming revenue typically retains between USD 55,000 and USD 70,000 after platform fees and distributor commissions — a figure that falls substantially lower for label-signed artists (Music Business Research, 2024). Centralized intermediaries retain between 30 and 60 percent of gross creator revenue, varying by platform and contract structure; the per-platform breakdown used in our analysis is presented in Section 7.2.3 (Table 7.12). Fees are non-negotiable, audiences are non-portable, and content rights remain bound by the platform’s terms of service throughout.

Consumers face analogous fragmentation. A film bought on iTunes cannot be streamed on Amazon Prime; an Audible audiobook cannot be transferred to Google Play; a subscription started on one platform cannot move to another. Consumers do not own their purchases; they hold a platform-bound license that can be revoked, modified, or ignored. The fragmentation that constrains creators also restricts the consumers who support them.

Fragmentation extends to monetization models. Subscriptions typically live on Patreon; one-time sales on Gumroad; pay-per-view is scarcely supported at all, because the overhead of conventional payment systems makes micro-transactions uneconomic. Each platform supports one monetization model, holds its own audience, takes its own fees, and produces its own accounting reports. A creator who wants to offer all three models is forced to run three or four platforms in parallel and reconcile them by hand.

Existing blockchain alternatives address parts of the problem but not the whole. Non-fungible tokens (NFTs) provide transferable ownership but not recurring access or per-view consumption. Subscription-style decentralized platforms (such as Audius)

offer recurring revenue but are confined to a single chain and audience. Cross-chain bridges move assets between ecosystems but lack rights metadata. Blockchain-based digital rights management has been examined in the academic literature specifically for the music industry (Ciriello et al., 2023), with proposed solutions sharing a common limitation: they address rights metadata or royalty distribution in isolation, without combining all three monetization models into a single portable primitive. No existing system, centralized or decentralized, delivers subscription, pay-per-view, and permanent ownership from a single rights primitive portable across blockchains.

The technical primitives to resolve this (cross-chain messaging, programmable smart contracts, on-chain royalty distribution, and NFT-based rights tokenization) all exist as standalone capabilities. Hanneke et al. (2024) characterize this class of systems as an emerging *Internet of Value*, in which blockchain tokenization enables asset ownership and revenue flows that bypass centralized intermediaries entirely. Our research examines whether these primitives can be integrated into a coherent system that benefits creators over existing solutions, and the conditions under which creators would adopt it.

1.2 | Background Context

This dissertation sits at the intersection of three research and engineering areas: blockchain-based digital rights management, cross-chain interoperability, and decentralized content monetization. Each is surveyed in Chapter 2; this section gives the context needed to motivate the work.

The global Digital Rights Management market is large and growing. Independent estimates from MarketsandMarkets, Mordor Intelligence, and Grand View Research place it at approximately USD 6.7 billion in 2025, forecast to reach USD 11–14 billion by 2030 (10–11% CAGR), driven by streaming services, OTT platforms, rising AI-related piracy, and regulatory pressure. The blockchain-specific segment is smaller but expanding at approximately 33% annually, projected to reach USD 2.74 billion by 2030.

Cross-chain interoperability has emerged as an active research area during the implementation window. A 2023 survey in IEEE Transactions on Knowledge and Data Engineering (Ren et al., 2023) classifies interoperability approaches into five categories — sidechains, notary schemes, hashed time lock contracts, relays, and blockchain-agnostic protocols — and two further 2025 surveys (Deng et al., 2025; Li et al., 2025b) collectively cover more than 150 sources. The Polkadot ecosystem that underpins our technical foundation completed its 2.0 upgrade during this period, introducing Async Backing, Agile Coretime, and Elastic Scaling. These features materially improve per-parachain latency

and throughput. Snowbridge, the trust-minimized Polkadot–Ethereum bridge, launched V2 in November 2025 with USD 75 million in total value locked. JAM, Polkadot’s next-generation relay-chain architecture, reached testnet in January 2026.

In January 2026, the ink! Alliance discontinued development of ink!, the language chosen for the CCRMS contract layer; our pallet-first architecture proved resilient to the change (Appendix A.9). The EU AI Act copyright provisions (August 2025) and MiCA (full enforcement July 2026) create regulatory demand for cryptographic provenance and tamper-evident rights records, functions that CCRMS can provide.

1.3 | Research Aim and Approach

Our objective is to **design, implement, and evaluate a functional prototype of a cross-chain content rights management service that supports three monetization models: subscription, pay-per-view, and permanent ownership, using a single unified rights primitive with automatic royalty distribution and cross-chain portability**. We designate this prototype as CCRMS (the Cross-Chain Content Rights Management Service) and implement it as a Polkadot parachain.

The primary research question that guides our work is:

How can a shared-security, multi-chain framework built on Polkadot’s Cross-Consensus Messaging (XCM) and FRAME pallet primitives deliver a unified rights token that natively supports recurring subscriptions, pay-per-view microtransactions, and permanent ownership transfers across heterogeneous blockchain networks, and what levels of transaction finality, cost efficiency, and creator revenue retention can such a system achieve?

The question comprises two distinct components: a **design feasibility** question (assessing whether the system can be constructed using available primitives) and a **measurement** question (determining the system’s operational properties and comparing them to alternative options). We treat these components as mutually constraining and report their results relative to one another. A comprehensive statement of the primary question, the five sub-questions that decompose it, and the six research objectives addressing them is presented in Chapter 3.

Our research approach is grounded in **design science research** (Hevner et al., 2004) as applied to a multi-component blockchain prototype. We organize the methodology, detailed in Chapter 4, into iterative build-test-measure-refine cycles within a six-phase framework. Our evaluation strategy, as delineated in Chapter 6, combines quantitative

measures, including technical benchmarks, security audits, comparative analyses, and Monte Carlo simulations of creator revenue, with qualitative insights from literature synthesis. We frame the evaluation as **measurement-focused** rather than target-driven: our objective is to document and analyze the levels of performance, security, and economic viability achieved by the system, rather than to verify conformity with a predefined set of criteria.

1.4 | Contributions

We make contributions across four categories, described in detail in Section 9.1:

1. **A working artifact:** the CCRMS parachain implementation (Rust, ink!, and JavaScript), released as open source under the MIT license.
2. **Formal documentation:** seven implementation specifications that explain what the artifact does and why each design decision was made.
3. **Empirical findings:** technical performance measurements, a Monte Carlo creator-revenue result (CCRMS produces higher mean revenue, strongly conditional on audience size), and eight security audit findings, of which one Medium and one Low were remediated.
4. **A worked example** of applying design science research to a multi-component blockchain prototype, a combination seldom exemplified in existing DSR literature.

1.5 | Scope and Limitations

This dissertation is bound in several ways that the reader should be aware of prior to engaging with the main content chapters.

The prototype is a back-end service, not a consumer application. We did not develop a web or mobile interface for CCRMS. Our assessment focuses on pallet- and runtime-level behavior; we do not directly evaluate user-experience factors. We recognize this as a limitation in Chapters 6 and 8 and as an area for future research in Chapter 9.

We take the measurements on a local development network. Our CCRMS prototype runs in a local Zombienet topology with two relay-chain validators and single-collator parachains. In production, parachains are hosted on the Polkadot or Kusama relay chains, which have hundreds of validators, real-world network latency, and competition from many other parachains. The throughput, latency, and reliability metrics in Chapter 7

should be treated as **upper bounds for the test environment**, not forecasts of real-world deployment performance.

Our Snowbridge integration uses a local Ethereum stack. We successfully demonstrated the bridge with two transactions, each for one ETH, bridging from a local Ethereum chain to a local AssetHub parachain. However, the entire configuration remains local; no public testnet or mainnet has been used. This integration serves as a proof of architectural feasibility rather than a production validation.

We have not commissioned any third-party audit, security review, or independent benchmarking. A single author performed the implementation, test design, measurements, and interpretation in this dissertation. Deployment to production would require third-party validation before any substantive value is processed.

We did not fully achieve two of the six research objectives. RO5 (zero-knowledge proof hooks for selective regulatory disclosure) was not undertaken. The complexity of integrating a ZK proof system into a Substrate runtime, selecting a proof system, building or adopting a verifier pallet, designing the selective disclosure protocol, and producing testable proof-of-concept circuits was substantially greater than the original Phase 2 objective anticipated. We acknowledge the underestimate. RO2 (off-chain indexing) was partially implemented: the pallet's emitted events are sufficient for direct on-chain queries, but we did not deploy a separate indexer (e.g., SubQuery). We document both omissions in Chapter 3, Section 3.3, and revisit them as future work in Chapter 9.

Background and Related Work

This chapter sets the academic and commercial context for the Cross-Chain Content Rights Management Service (CCRMS). It synthesizes a broad range of scholarly and industry publications into a coherent narrative organized around four primary themes: blockchain-based content management (Section 2.1), cross-chain interoperability (Section 2.2), decentralized monetization models (Section 2.3), and the challenges and gaps that motivate our research (Section 2.4). The chapter draws on peer-reviewed academic publications, industry reports and market research, and primary technical documentation from the Polkadot ecosystem.

2.1 | Blockchain-Based Content Management

2.1.1 | Market Context

Three independent market research firms (Grand View Research, 2025; MarketsandMarkets, 2025; Mordor Intelligence, 2026) converge on estimates that the global DRM market is valued at approximately USD 6.7–6.9 billion in 2025–2026, with projections reaching USD 11–14 billion by 2030–2033, representing a Compound Annual Growth Rate (CAGR) of 10–11%. This growth is primarily driven by streaming services, AI-enabled piracy, and regulatory pressures. The market is dominated by **content protection** solutions (such as Widevine, FairPlay, PlayReady), rather than **content monetization**. The growth opportunity for blockchain-based alternatives lies in bridging the gap between protection and monetization.

2.1.2 | Foundational Work on Blockchain-Based DRM

Blockchain-based Digital Rights Management (DRM) is an active research domain. Garba et al. (2021) incorporate digital watermarking into blockchain technology as a tamper-evident layer; Patil et al. (2022) build on this by introducing the DRMChain hybrid encryption architecture; and Yi et al. (2023) further extend the protection model with a redactable blockchain scheme that combines DRM with perceptual hashing, enabling selective content modification without invalidating ownership proofs. Both earlier studies prioritize protection mechanisms over the economics of content creators. Furthermore, Singh et al. (2024b) and Tao (2025) survey blockchain-based DRM applications in library contexts; MDPI's *Information* (Information (MDPI), 2026) proposes a comprehensive library DRM framework that integrates the InterPlanetary File System (IPFS) with smart contracts.

For the music industry, Ciriello et al. (2023) derive a set of design principles for blockchain-based DRM systems through expert interviews and prototype testing, identifying transparency of rights ownership, automated royalty distribution, and programmable licensing as the three foundational requirements. Cherdakov et al. (2023) and Mendoza-Tello et al. (2024) propose Web3 implementations that embed these principles in music streaming smart contracts, demonstrating the feasibility of on-chain licensing but stopping short of cross-chain portability.

The most operationally comprehensive recent work is Madapati and Pradhan (2025), whose Ethereum + IPFS + perceptual hashing framework achieves 409 TPS for registration and 484 TPS for verification, both higher than CCRMS. However, this is misleading because their pipeline hashes off-chain and stores only results on-chain. Wang et al. (2025) present a BLS threshold signature scheme for intellectual property transactions that achieves 40 ms aggregation. Two Polkadot-specific studies (Xie and Liu, 2022; Xie and Tang, 2024) propose conceptual cross-chain copyright frameworks without working prototypes.

The collective contribution of this literature is that **blockchain-based DRM is a legitimate research domain with growing operational maturity**; its limitation is that nearly all work emphasizes protection rather than monetization. CCRMS explicitly reverses this priority.

2.1.3 | NFTs and Intellectual Property Rights

Work on NFTs in the context of intellectual property encompasses both legal analysis (Davik, 2025; Eurojust, 2024; Murray, 2022; USPTO, 2024) and proposals for standard-

ization (Dreyer et al., 2021; Gatto et al., 2021; Reggianini, 2025). The most empirically significant study is Darshan et al. (2025), whose findings strongly support our research: **80% of mainstream NFTs fail to transfer copyright to purchasers, only 12–30% of marketplace listings include licensing information, and 50–60% of secondary sales bypass creator royalties.** Lahiri et al. (2026) provide an independent meta-analysis of 125 NFT studies, confirming these findings and concluding that legal clarity around NFT ownership transfers remains the field’s primary unsolved problem. The existing NFT infrastructure is theoretically capable of effective rights management but is inadequate in practice. Cabot-Nadal et al. (2022) address a specific gap in the ERC-721 standard, proposing an extension that enables token recipients to reject unwanted transfers — a property relevant to our permanent-ownership mode, where rights revocation must be cryptographically blocked.

The most significant industry development is the February 2025 launch of **Story Protocol**, the first dedicated intellectual-property Layer-1 blockchain (Story Foundation, 2025). Built on the Cosmos SDK with EVM compatibility and backed by USD 140 million in total funding (including a USD 80 million a16z-led Series B), it enables programmable IP registration, automated licensing, and on-chain royalty distribution. Story Protocol is the most direct industry comparator to CCRMS, validating the market premise and offering a useful architectural contrast: single-chain L1 versus cross-chain parachain.

2.2 | Cross-Chain Interoperability

2.2.1 | Polkadot Architecture

Polkadot (Wood, 2016) is a heterogeneous multi-chain framework in which parachains share security via a central relay chain and communicate via XCM. The core primitives are parachains, the relay chain, validators, collators, and XCM. Academic analyses of Polkadot (Abbas et al., 2022; Moreno et al., 2023) treat it as a research-worthy architecture but do not address content rights management. Habermeier et al. (2021) and Wood (2022) provide the primary technical documentation for the XCM message format and multi-hop features that underpin our cross-chain implementation. The XCM protocol itself has received dedicated academic attention: Morháč et al. (2023a) build ParaSpell, a developer SDK that abstracts XCM message construction across parachains, and Morháč et al. (2023b) demonstrate XCMP interoperability at the protocol level. Together with Morháč et al. (2025), who propose the Unispell universal adapter, they form the most coherent body of peer-reviewed academic work on XCM in practice.

2.2.2 | Cross-Chain Technology Surveys

The cross-chain literature matured rapidly from 2022 to 2025. Ren et al. (2023) provide a comprehensive taxonomy in IEEE Transactions on Knowledge and Data Engineering, classifying interoperability solutions into five categories: sidechains, notary schemes, hashed time lock contracts, relays, and blockchain-agnostic protocols. Mao et al. (2022) covered the same terrain earlier with a three-category classification; Bigiotti et al. (2025) extend both frameworks in ACM Computing Surveys, with particular attention to protocol-agnostic designs. Two more recent surveys significantly expand the field: **Deng et al. (2025)** systematically analyze more than 150 sources covering atomic swaps, sidechains, light clients, and relay chains; **Li et al. (2025b)** distinguish between **asset interoperability** (the transfer of tokens across chains) and **data interoperability** (the verification of cross-chain state). Li's distinction is particularly relevant to our research, which addresses both aspects through XCM payments and the pallet-rights-verifier Merkle proof mechanism. Kumar et al. (2025) examine cross-chain transaction efficiency for future internet-of-value applications, a framing directly aligned with CCRMS's design intent; Cao et al. (2025) propose the MAP protocol as a trustless, scalable interoperability framework, evaluated at the ACM Web Conference 2025. Sevim (2024) provides comparative metrics across LayerZero, Wormhole, Chainlink, Circle, Polkadot, and Cosmos; Valaštín et al. (2024) introduce LiquiSpell for cross-parachain liquidity, and their observation of parachain heterogeneity informs our cross-chain operational pattern.

2.2.3 | Bridges and Snowbridge

The academic literature on bridge architectures includes Yin et al. (2022) on Bool Network (multi-party computation over hidden committees), Chen et al. (2023) on reputation-based trust management, and Maric et al. (2025) on formal verification of fail-safe bridges using Isabelle/HOL. Maric et al.'s research sets a methodological precedent for future formal analyses of CCRMS itself.

The most operationally significant development in blockchain infrastructure is **Snowbridge**, a trust-minimized bridge that enables interoperability between Polkadot and Ethereum. Snowbridge V2 was officially launched on the Polkadot mainnet on November 7, 2025, and includes features such as arbitrary Ethereum contract calls from Polkadot, a 50% fee reduction, unordered message execution, transfer times of approximately 35 minutes, and a total locked value of around USD 75 million at inception (Snowfork, 2025a). The associated Polkasassembly proposal (Snowfork, 2025b) documents more than a year of operational stability with no on-chain downtime. We position our Snowbridge in-

tegration as forward-compatible with V2, even though we developed the initial prototype against V1.

2.2.4 | Polkadot 2.0 and JAM

Two significant ecosystem developments shaped our implementation timeline. **Polkadot 2.0** was comprehensively launched by October 2025, introducing Async Backing (eliminating relay-chain wait), Agile Coretime (on-demand block production), Elastic Scaling (multiple cores per parachain), and XCM v5 (multi-hop transactions) (Parity Technologies, 2025; Polkadot Network, 2025). Parachains using the full feature set achieve approximately 2-second block latency with three cores, with an anticipated reduction to 500 milliseconds at twelve cores. **JAM** (Join-Accumulate Machine, Wood (2024)) is expected to succeed the relay-chain model within the Polkadot ecosystem. Its testnet launched in January 2026, featuring 43 implementation teams competing for a prize of 10 million DOT, with the mainnet scheduled for mid-to-late 2026. These developments collectively demonstrate that the Polkadot architecture is **maturing rather than stagnating**, creating a positive environment for ongoing research within the ecosystem.

2.2.5 | Cross-Chain Copyright Management as a Research Target

Numerous studies examine cross-chain copyright management as a research issue (Buzu, 2021; Mareckova, 2024; Munson et al., 2022; Orero et al., 2023; Xie and Liu, 2022; Xie and Tang, 2024); however, none have developed a functional end-to-end prototype comparable to the CCRMS platform.

2.3 | Decentralized Monetization Models

2.3.1 | Creator Economy Context

CommuniPass (2026) forecasts that the creator economy will exceed USD 280 billion by the end of 2026. The blockchain-specific segment is relatively small but growing rapidly: 360iResearch (2025) predicts it will be worth USD 499.52 million in 2024, rising to USD 2.74 billion by 2030, a CAGR of 32.78%. The primary drivers of this growth include creator dissatisfaction with centralized platforms, demand for cross-platform interoperability, and increasingly mature blockchain infrastructure, all of which align with the gaps that CCRMS aims to address.

2.3.2 | DeFi and Web3 Foundations

The conceptual foundation for decentralized monetization lies in DeFi. Gogel et al. (2021) present the canonical Wharton overview of the DeFi ecosystem, covering AMMs, lending, derivatives, and stablecoins; Atzori (2017) analyzes decentralized governance through a political-economy lens; and Busch (2022) provides a technical overview of Web3 from the United States Congressional Research Service. The congressional context affirms that Web3 has transitioned from a niche technological movement to a policy concern for major governments.

2.3.3 | Subscription, Pay-Per-View, and Purchase Models

Earlier work on blockchain subscription and pay-per-view models includes Banerjee et al. (2020); Castweet Team (2019); Khan (2024); Munson (2019); Steem Team (2025). Most of these works examine blockchain incentives without addressing the micropayment problem. Two more recent works are particularly relevant: **Osemwegie (2025)** surveys decentralized media monetization (identifying the problems CCRMS targets but not proposing a working system), and **Shah (2025)** proposes a smart-contract framework for royalty distribution that is the closest to our royalty propagation design in the existing literature. CCRMS extends Shah's approach in three respects: royalty distribution is one feature within a broader unified rights token rather than the central contribution; the implementation uses a FRAME pallet with atomic storage rollback rather than a Solidity contract with weaker atomicity guarantees; and the design supports up to 10 collaborators with basis-point precision, validated through unit tests and benchmarks. Wamugo (2024) provides empirical data on NFT adoption by media organizations in Kenya, demonstrating that blockchain monetization is under active consideration in emerging markets.

2.3.4 | Pay-Per-View and the Micropayment Problem

Pay-per-view models have historically struggled on blockchain platforms because gas fees have often exceeded content costs. Goyal et al. (2019) introduce Gringotts, a secure micropayment system, but do not address the core fee issue. Chegenizadeh et al. (2024) directly address the micropayment verification layer, proposing a multi-agent, privacy-preserving approach for resource-constrained offline devices that reduces per-transaction overhead through smart-contract batching; their work supports our strategic decision to model PPV as a counter rather than a sequence of individual on-chain transactions. A notable industry advancement is the **Coinbase x402 protocol** (Coinbase and x402

Foundation, 2025) (May 2025), which embeds payment logic directly in HTTP responses, enabling stablecoin settlement in two seconds. The x402 deployment affirms our strategic decision to conceptualize pay-per-view as a counter rather than a sequence of individual transactions.

2.3.5 | Smart Contracts as the Implementation Layer

Surveys of the smart contract literature (Ante, 2020; Lin et al., 2022; Quan et al., 2024; Singh et al., 2024a) establish the field as mature. Parity Technologies' ink! blog posts (Parity Technologies, 2022a,b) served as the primary technical references during our implementation, when ink! was the recommended smart contract language for Polkadot parachains. The most consequential development in this field is the **discontinuation of ink! development in January 2026** (ink! Alliance, 2026), which we discuss as a literature gap (G8 in Table 2.1) and as an architectural episode in Appendix A.9. A related development is Vitalik Buterin's April 2025 proposal to replace Ethereum's EVM with RISC-V (Buterin, 2025), which validates Polkadot's earlier transition to PolkaVM and indicates a broader industry convergence toward RISC-V.

2.3.6 | Royalty Automation

Shah (2025) and Reggianini (2025) directly address the automation of royalties, although most literature on this topic appears in industry blog posts and white papers rather than in peer-reviewed journals. Our royalty propagation design (Section 5.2.3) aligns with Shah's methodology, which uses a basis-point distribution with atomic on-chain enforcement, and extends this approach by supporting up to 10 collaborators per content item and by using FRAME storage rollback to provide stronger atomicity guarantees than those of Solidity contracts.

2.4 | Challenges and Gaps

We synthesize the literature above into eight gaps that CCRMS addresses. Table 2.1 summarizes each gap and our corresponding contribution.

Table 2.1. Literature Gaps and CCRMS Contributions

ID	Gap	Supporting literature	CCRMS contribution
G1	Existing systems are protection-focused, not monetization-focused	Garba et al. (2021); Patil et al. (2022); Ciriello et al. (2023); Madapati and Pradhan (2025); commercial DRM (Widevine, FairPlay, PlayReady)	Unified rights token with monetization as the primary design goal; protection becomes a secondary consequence of correct rights management.
G2	No native cross-chain recurring payments	Existing cross-chain protocols (XCM, Wormhole, LayerZero, Snowbridge) address one-time transfers; Xie and Liu (2022); Xie and Tang (2024) propose theoretical frameworks without recurring payments	On-chain <code>on_initialize</code> block hook scheduler for automatic renewal. XCM v5 Schedule is not yet production-ready; we defer true cross-chain scheduling to future work.
G3	Cross-chain protocols strip rights metadata on transfer	Reggianini (2025) addresses part of the standardization problem within European NFT licensing	<code>query_rights_metadata</code> extrinsic emitting a self-describing <code>RightsMetadata</code> event; metadata canonical on CCRMS, authenticated by <code>pallet-rights-verifier</code> Merkle proofs.
G4	Pay-per-view is economically unviable at micro-transaction scale on traditional blockchains	Goyal et al. (2019); Chegenizadeh et al. (2024); Khan (2024); x402 (Coinbase and x402 Foundation, 2025)	PPV reconceptualized as a counter: one payment per pack, cheap storage updates per view, with additive purchases (Section 5.2.4).

Continued on next page

Table 2.1 continued from previous page

ID	Gap	Supporting literature	CCRMS contribution
G5	The decentralization–latency trade-off is poorly characterized quantitatively	Most works endorse decentralization qualitatively or measure only centralized performance; Madapati and Pradhan (2025) is an exception but uses an off-chain pipeline; Rao et al. (2024) provide a scalability review but no cross-paradigm comparison	Direct comparative benchmark against an Express.js + SQLite baseline (Section 7.1.7) plus HHI calculation (Section 7.3.3) make the trade-off numerical.
G6	Regulatory uncertainty for tokenized rights (EU AI Act Aug 2025; MiCA full enforcement Jul 2026; EP report A10-0019/2026)	European Securities and Markets Authority (2025); Guadamuz (2025)	Cryptographic provenance, immutable records, and automated royalty distribution align with emerging compliance infrastructure. MiCA classification of subscription tokens remains an open legal question.

Continued on next page

Table 2.1 continued from previous page

ID	Gap	Supporting literature	CCRMS contribution
G7	No existing system provides first-class native support for subscription, pay-per-view, and permanent ownership as three access modes of a single unified rights primitive . Story Protocol's Programmable IP License (PIL) can be configured to approximate these models, but requires separate PIL configurations and license-token types for each, and does not natively model time-bounded subscription renewal.	Patreon (subscription only); OpenSea (ownership only); pay-per-view absent from most blockchain systems; Story Protocol (flexible licensing primitive, not unified rights); Shah (2025) addresses royalty automation as a single feature	Unified rights token treats the three models as native access modes of a single content registration, not as configurations of a generic licensing primitive. A single FRAME pallet storage entry (Content<T>) encodes subscription terms, PPV quota, and ownership state simultaneously; one integrated access check enforces all three; renewal and expiry are first-class state transitions, not contract-layer constructions. This is the dissertation's central conceptual contribution.
G8	ink! discontinuation (January 2026) leaves Polkadot without a viable institutionally supported smart contract language; alternatives (revive, raw pallet-revive, wrevive) each carry trade-offs	Appendix A.9	Pallet-first architecture demonstrates that substantial functionality can be delivered without a smart contract language. The gap remains for systems whose business logic is better expressed in contracts.

We fully address gaps G1 and G7; we address G6 and G8 only partially because they depend on factors outside the prototype's control. The collective contribution is that a single working prototype demonstrates feasibility for a category of systems that the literature has addressed only partially.

2.5 | Summary

We reviewed 100 academic and industry sources on blockchain-based content management, cross-chain interoperability, and decentralized monetization. The foundational primitives are in place; the market for decentralized content monetization is real and expanding rapidly; and existing methodologies address the issue only partially. CCRMS bridges the eight identified gaps by deploying a unified rights token on a Polkadot parachain.

Research Questions and Objectives

This chapter states the research questions and objectives that underpin our study. We formulated them during Phase 1 and refined them in Phase 2 after the gap analysis of Chapter 2.

3.1 | Primary Research Question

The primary research question of our work is:

How can a shared-security, multi-chain framework built on Polkadot's Cross-Consensus Messaging (XCM) and FRAME pallet primitives deliver a unified rights token that natively supports recurring subscriptions, pay-per-view micro-transactions, and permanent ownership transfers across heterogeneous blockchain networks, and what levels of transaction finality, cost efficiency, and creator revenue retention can such a system achieve?

The question has two components: **design feasibility** (whether such a system can be built using the available Polkadot SDK primitives) and **measurement** (what operational properties it exhibits once built). We treat them as mutually constraining; a design that is feasible to construct yet demonstrates subpar performance is not a viable solution, nor is one that performs well in benchmarking but cannot be implemented in practice. The primary concern is framed as an **achievability** question rather than a hypothesis test, consistent with the measurement-focused evaluation philosophy established in Section 4.1.4.

3.2 | Sub-Questions

We decompose the primary research question into five sub-questions. Table 3.1 presents each with its motivation and the literature gaps it addresses.

Table 3.1. Research Sub-Questions

ID	Question	Gaps	Motivation
SQ1	To what extent can XCM carry recurring subscription renewal messages and rich rights metadata across Polkadot parachains and external networks, and what atomic success rates and cross-chain finality times are achievable?	G2, G3	Existing cross-chain protocols lack native recurring payments and strip rights metadata. SQ1 asks whether existing XCM primitives suffice or whether custom extensions are needed.
SQ2	What design patterns can minimize access verification latency while maintaining decentralization, and how do the resulting metrics compare across varying validator configurations?	G5	The decentralization-latency trade-off is poorly characterized quantitatively. SQ2 asks for a direct measurement on a working Polkadot parachain with explicit numerical comparisons against centralized alternatives.
SQ3	Can a single on-chain rights object simultaneously enforce subscription auto-renewal, pay-per-view consumption limits, and permanent ownership with automated royalty distribution across chains, and what are the resulting creator cost savings?	G7	No existing system offers all three monetization models as first-class access modes of a single unified primitive. Story Protocol's PIL can be configured to approximate them but requires separate license-token types and does not natively model subscription renewal. SQ3 asks whether a unified primitive can be built and whether its economics justify the effort.

Continued on next page

Table 3.1 continued from previous page

ID	Question	Gaps	Motivation
SQ4	What authorization model is appropriate for cross-chain rights operations that decouple the payer from the beneficiary, and how does it perform under adversarial conditions?	(added Phase 5)	Added in response to Finding B from the security audit. Cross-chain extrinsics decoupling payer from beneficiary pose unique authorization concerns absent from local-only extrinsics.
SQ5	Under realistic creator audience distributions, does the CCRMS model produce demonstrably better creator outcomes than centralized, bridge-based, or marketplace alternatives, and under what conditions?	(added Phase 5)	Added during the Monte Carlo simulation. SQ5 clarifies that the economic question is conditional (“yes, for creators with these characteristics”) rather than a simple yes-or-no.

3.3 | Research Objectives

We pursue the five sub-questions through six research objectives. Table 3.2 summarizes each; Appendix B provides the full statements, deliverables, and revision notes.

Table 3.2. Research Objectives Summary

Objective	Focus	Sub-questions
RO1	Design and prototype XCM extensions for recurring and metadata-rich rights transfers	SQ1, SQ2
RO2	Implement a unified rights token (originally ink!, revised to FRAME pallet) with access-verification latency measurement	SQ2, SQ3
RO3	Merge subscription, PPV, and purchase logic into one composable rights pallet	SQ3, SQ4

Continued on next page

Table 3.2 continued from previous page

Objective	Focus	Sub-questions
RO4	Benchmark scalability, interoperability, and economic efficiency against centralized and bridge-based alternatives	SQ2, SQ5
RO5	Incorporate optional zero-knowledge proof hooks for selective regulatory disclosure (restated as future work in Section 9.3)	Broader feasibility
RO6	Quantify creator savings via Monte Carlo simulation and comparative cost analysis	SQ5

3.4 | Mapping Questions to Objectives, Methods, and Chapters

Table 3.3 maps each sub-question to its research objectives, evaluation methods, and the sections where results are reported.

Table 3.3. Mapping of Sub-Questions to Objectives, Methods, and Chapters

Sub-Question	Primary Objective	Supporting Objective(s)	Primary Method	Reported In
SQ1 (XCM rights operations)	RO1	RO3	XCM latency benchmark; end-to-end XCM test	Section 5.3; Section 7.1.2; Section 7.1.6
SQ2 (Latency vs decentralization)	RO2, RO4	–	Throughput and latency benchmark; HHI calculation	Section 7.1.2; Section 7.3.3
SQ3 (Unified rights token)	RO3	RO2, RO6	Unit tests; royalty propagation tests; Monte Carlo simulation	Section 5.2; Section 7.2

Continued on next page

Table 3.3 continued from previous page

Sub-Question	Primary Objective	Supporting Objective(s)	Primary Method	Reported In
SQ4 (Security and authorization)	(added Phase 5)	RO3	Security audit; authorization unit tests	Section 5.4; Section 7.3.1
SQ5 (Economic viability)	RO6	RO4	Monte Carlo simulation; comparative benchmark	Section 7.2; Section 7.1.7

Two cells in the table need acknowledgment. The “Primary Objective” for SQ4 is marked “(added Phase 5)” because of a sub-question added after the original objectives were finalized. There is no direct Phase 2 link; however, RO3 (unified rights pallet) provided context for the security finding. RO5 (zero-knowledge proof hooks) is not in the table because it was not implemented, but it is kept in Section 3.3 (RO5) as an unachieved future objective and reiterated in Chapter 9.

3.5 | Summary

This chapter has presented the primary research question, its five sub-questions, and the six research objectives aligned with them. The questions are founded upon the gaps identified in Chapter 2. Table 3.1 provides an explicit mapping from each subsidiary question to the objectives it aims to address, the methodologies used to gather evidence, and the chapters in which the work is documented.

Two of the sub-questions (SQ4 on security and authorization, and SQ5 on economic viability) were incorporated during Phase 5 in response to findings from implementation and evaluation. This is consistent with the iterative prototyping methodology and exemplifies the principle of design science research, whereby the search process itself generates knowledge that may modify the initial framing of the work.

Methodology and System Design

This chapter outlines the research methodology we used to develop CCRMS and the resulting system design. Section 4.1 presents the methodology, including design science framing, iterative prototyping, data collection, and evaluation philosophy. Section 4.2 describes the system design at the conceptual level; Chapter 5 covers implementation details.

4.1 | Research Methodology

4.1.1 | Design Science Research Framing

CCRMS is fundamentally a systems-building project with a primary artifact: a working software prototype. The most appropriate methodological framework is therefore **design science research (DSR)**, as defined by Hevner et al. (2004). Our dissertation maps to Hevner’s seven DSR guidelines as follows: (1) *design as an artifact*: the CCRMS parachain and seven implementation specifications; (2) *problem relevance*: the commercial and academic gaps identified in Chapter 2; (3) *design evaluation*: eleven KPIs covered in Chapters 6–7; (4) *research contributions*: the artifact, the design knowledge captured in the specifications, and the empirical measurements; (5) *research rigor*: established Polkadot SDK primitives, standard Rust testing, and peer-reviewed Monte Carlo methodology; (6) *design as a search process*: nine significant pivots documented in Section 5.4; (7) *communication of research*: this dissertation and an open-source code release.

DSR is particularly appropriate here because it recognizes both the artifact and the design knowledge it generates as coequal contributions.

4.1.2 | Iterative Prototyping

Within the DSR framework, the development process follows an **iterative prototyping** model (build, test, measure, refine). We chose iteration deliberately: the Polkadot SDK monorepo releases stable versions approximately quarterly, making a waterfall approach prone to conclusions based on an outdated toolchain, and the architectural intricacies of a cross-chain system mean that certain design decisions can only be thoroughly assessed once the system is operational. In practice, this iterative methodology led to nine substantial architectural pivots during Phase 4, documented as challenges in Section 5.4. We executed the work across six four-week phases (Appendix F); references to “Phase N” throughout the dissertation denote work produced in that phase.

4.1.3 | Data Collection Methods

Our evaluation uses a mixed-methods approach: quantitative measurements include unit tests, throughput and latency benchmarks, storage growth, cross-chain XCM latency, reliability tests, a centralized comparative benchmark, and a Monte Carlo simulation of creator revenue. Qualitative inputs are drawn from a literature synthesis. Chapter 6 describes each method. All quantitative outputs are systematically stored as machine-readable JSON files in the `scripts/perf/results/` directory to ensure reproducibility.

4.1.4 | Evaluation Philosophy: Measurement-Focused, Not Target-Driven

We adopt a **measurement-focused philosophy**: the goal is to document and analyze the system’s performance, efficiency, security, and economic viability, rather than to verify that it conforms to predetermined targets. This measurement-centric approach preserves academic rigor regardless of whether specific results meet hypothetical benchmarks, treats unexpected results as valuable rather than embarrassing, encourages truthful reporting, and establishes a baseline for future researchers.

Each of the eleven KPIs in Section 6.5 has a target derived from the Phase 2 methodology, but we treat those targets as reference points rather than pass/fail criteria. In practice, all eleven met their targets; the methodology would have remained valid even if some had not.

4.1.5 | Methodology Pivots

We revised the methodology established in Phase 4. The original plan called for Kusama via the Canary Chaos program, implementing RMRK 2.0 composable NFTs, scheduling XCM for automatic renewal, and conducting a Monte Carlo simulation, which had not yet been executed. The actual implementation modified this approach by substituting Kusama with local Zombienet to enable faster iteration, replacing RMRK 2.0 with `pallet-nfts` (noting that the RMRK pallets are frozen on SDK version 0.9.36), replacing the scheduled XCM with an `on_initialize` block hook (since the XCM v5 `Schedule` instruction is not yet production-ready), and completing the Monte Carlo simulation in Phase 5.

The most significant methodological pivot is unrelated to the original plan: the **emphasis on architectural decisions and pivots as first-class research outcomes**. The Phase 2 methodology treated implementation merely as a means to generate measurements; however, in practice, the architectural decisions made during the pivots proved to be equally valuable contributions as the measurements themselves.

4.2 | System Design

This section outlines the conceptual design. Section 5.2 provides the implementation details.

4.2.1 | The Unified Rights Token Concept

This subsection addresses SQ3: it demonstrates that subscription, pay-per-view, and permanent ownership can serve as first-class access modes for a single on-chain record rather than as separate platform categories.

The principal intellectual contribution of CCRMS is the **unified rights token**: a single on-chain asset that can represent three distinct modes of content access (an active subscription, a remaining pay-per-view balance, or permanent ownership) for a single piece of content from a single creator, all governed by a unified set of rules.

The unified token addresses the gap identified as G7 in Section 2.4. Current blockchain platforms force creators and consumers to choose among monetization models: subscription platforms cannot represent per-view or ownership rights, NFT marketplaces cannot enable recurring access, and pay-per-view systems rarely work effectively on-chain. The unified token treats the three models as **access modes** of a single underlying content registration, rather than as separate platforms. A creator registers content once and can

offer it through all three modes concurrently; a consumer chooses the mode that best suits their needs; the system enforces all three from a single canonical record. This approach eliminates a source of friction in the market: creators no longer need to maintain three distinct platform accounts for different audiences, and consumers no longer have to navigate three separate interfaces with separate payment flows.

4.2.2 | The Three Monetization Models

The three monetization models address SQ3 (unified primitive) and SQ5 (economic viability). Each model targets a distinct creator-economy use case, and the combined offering positions CCRMS against the centralized alternatives evaluated in Chapter 7.

The architecture supports three models, each motivated by a different gap in the existing landscape:

- **Subscription:** time-limited access valid for a configurable number of blocks per content item, with optional automatic renewal driven by an on-chain block hook.
- **Pay-per-view (PPV):** prepaid access to a specified number of views of a content item is available. Each `consume_view` operation decrements the counter; additional purchases increase the count rather than overwrite it. Conceptualizing PPV as a counter rather than a sequence of individual view transactions avoids the micro-transaction problem that has historically impeded on-chain PPV models — an approach corroborated by Chegenizadeh et al. (2024), who demonstrate that batching micropayment verification through smart contracts substantially reduces per-transaction overhead on resource-constrained devices.
- **Permanent ownership:** unrestricted and transferable access. Ownership is prioritized during the access verification process. Transfers follow a mint-burn-mint pattern rather than a direct NFT transfer, preserving provenance and attributive consistency.

A single content item can offer all three models concurrently with independently configured pricing. The `check_access` function resolves rights in priority order (Ownership > Subscription > PPV).

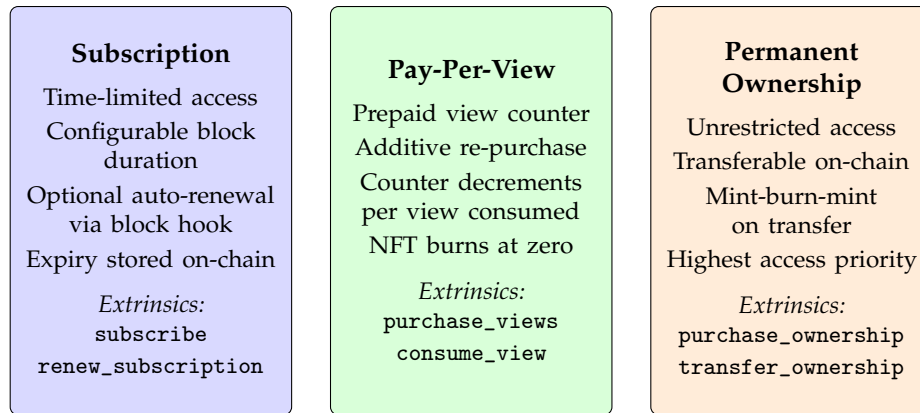


Figure 4.1. The three monetization models natively supported by the unified rights token (addresses SQ3). A single content registration may offer all three models concurrently; `check_access` resolves in priority order: Ownership > Subscription > PPV.

4.2.3 | NFT-Based Rights Tokenization

NFT-based tokenization provides the technical foundation for SQ3: standard `pallet-nfts` primitives can encode RMRK 2.0 composability semantics for a unified token without relying on unmaintained pallets.

CCRMS encodes content rights as non-fungible tokens using a parent-child nesting model built on `pallet-nfts`. The decision to use NFTs rather than a bespoke storage configuration was motivated by their widespread acceptance as a standard, compatibility with existing digital wallets, independent transferability (which facilitates a secondary market for ownership rights), and the clear separation of the content item's identity (the parent) from user-specific rights derived from it (the children). The mint-burn-mint transfer pattern for the permanent ownership mode is consistent with the approach proposed by Dondjio and Kazamias (2024), who demonstrate that re-minting on transfer preserves on-chain provenance without the ambiguity of direct NFT reassignment.

The conceptual model: each content registration is an NFT collection comprising one parent Content NFT and zero or more child rights NFTs, each tagged with a `rights_type` attribute (Subscription, PPV, or Ownership) and owned by the respective user. The parent-child structure serves three concurrent purposes: establishing identity, tokenizing individual rights, and enabling on-chain auditability of the rights-to-content relationship.

The nesting pattern is inspired by RMRK 2.0, the composable NFT specification for Polkadot; however, it is implemented directly on top of `pallet-nfts` rather than relying on the `rmrk-substrate` pallets, which are locked to SDK 0.9.36. Appendix A.1 provides a detailed account of this transition. The outcome shows that RMRK-style composability

can be achieved solely through standard FRAME primitives.

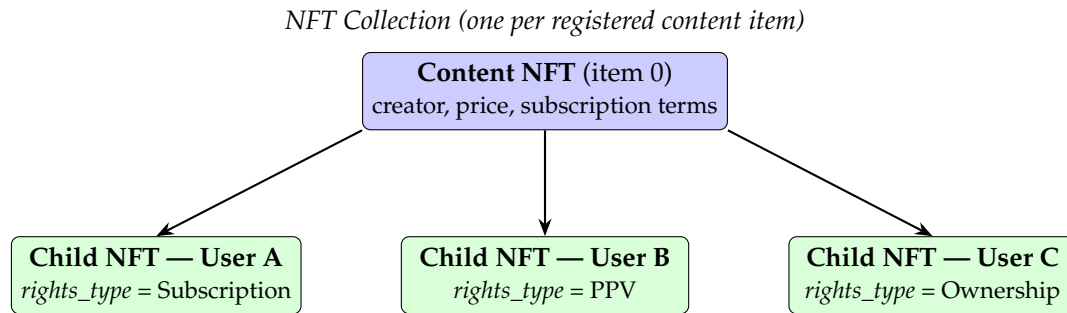


Figure 4.2. Parent-child NFT nesting model. Each content registration creates one NFT collection containing a parent Content NFT (the canonical on-chain identity of the content) and zero or more child rights NFTs, each owned by an individual user and tagged with their access mode.

One significant constraint is that the Children storage entry is a `BoundedVec<_, ConstU32<50>>`, limiting child NFTs to fifty per parent. This restriction is necessary for FRAME benchmarking, because the worst-case weight must be calculable in advance. The examination of this constraint is detailed in Appendix A.8, and mitigation is treated as future work in Section 9.1.

4.2.4 | Cross-Chain Communication Model

This model addresses SQ1 (XCM rights operations across chains) and SQ4 (payer-beneficiary authorization): the two properties that make cross-chain rights management nontrivial are the decoupling of payment from rights receipt and the trustless verification of remote rights state.

Cross-chain operation is the mechanism by which the unified rights token model enables portable digital ownership: a user who acquires rights on one blockchain should be able to use them on another without forfeiting access or repurchasing. The model has two conceptual properties.

Payer-beneficiary decoupling. Each cross-chain rights operation distinguishes between the funding account and the rights recipient account. In a local call, the payer and the beneficiary are the same (the caller); in a cross-chain call, the dispatch origin is the sovereign account of the remote parachain, and the beneficiary is a separate user account. This configuration enables a remote parachain to acquire rights on behalf of its users without requiring their presence on CCRMS.

Trustless verification of remote rights. A second pallet, `pallet-rights-verifier`, provides Merkle proof verification of the CCRMS rights state for other parachains. A

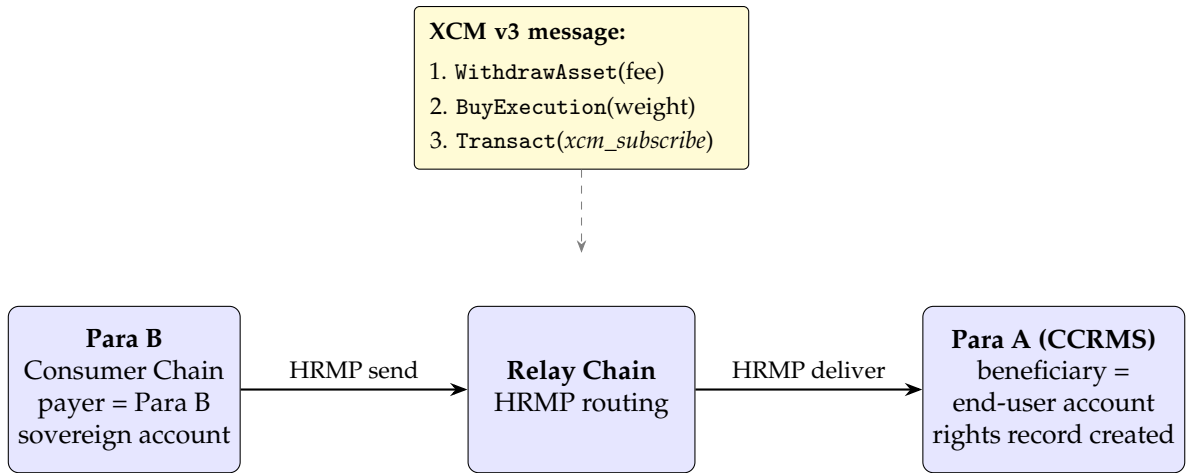


Figure 4.3. XCM cross-chain rights operation (addresses SQ1 and SQ4). The three-instruction sequence dispatches an `xcm_*` extrinsic on the CCRMS parachain, decoupling the paying account (Para B sovereign account) from the rights beneficiary (the end user). The same pattern applies to all five `xcm_*` extrinsics.

consuming chain can verify, without relying on any oracle or relay, that a specific account currently holds an active subscription by submitting a storage proof against a known CCRMS state root. This approach upholds the principle that CCRMS rights are **canonical on the CCRMS chain**: other chains authenticate cryptographically rather than maintaining replicas.

The model also extends beyond Polkadot via **Snowbridge**, a trust-minimized bridge connecting Polkadot and Ethereum. We configure the runtime to accept messages from Ethereum using the standard `LocationToAccountId` converters. At this stage, our architectural choice was to guarantee **forward compatibility with Ethereum bridging without relying on it**. Section 5.3 provides a comprehensive account of the cross-chain integration.

4.2.5 | Pallet-First Architecture

The pallet-first decision directly shapes the answers to SQ2 (latency versus decentralization) and SQ3 (unified primitive): placing business logic in a FRAME pallet rather than in smart contracts reduces execution overhead and eliminates the chain-extension boundary that would otherwise limit write operations.

The most significant architectural decision was to centralize the business logic within a **FRAME pallet** rather than in smart contracts. This decision was made during Phase 4 for three reasons: the operational maturity gap between FRAME pallets and `pallet-revive`;

the unavailability of the chain extension mechanism, which the original contract-first approach relied on, for write operations; and a pallet-first design with standard Polkadot wallets, without EVM-style adaptation. Appendix A.2 provides detailed documentation of this decision, while Appendix A.9 notes that it was unexpectedly validated by the discontinuation of ink! development in January 2026. The overarching principle — that architectural choices that reduce dependency on any non-essential component tend to yield benefits during ecosystem-level changes — is reflected in several other decisions within CCRMS.

4.2.6 | Layered Architecture

The layered architecture determines the answers to SQ1 (cross-chain reach) and SQ2 (the latency–decentralization trade-off): each layer can evolve independently, so XCM protocol upgrades and relay-chain migration to JAM do not require changes to the business logic in Layer 3.

The system is organized into six distinct, independently replaceable layers: (1) the **Polkadot relay chain** provides shared security and enables HRMP message passing; (2) the **parachain runtime** composes the CCRMS FRAME pallets, configures the XCM executor, and includes a custom EVM precompile; (3) the **business logic** layer comprises pallet-content-rights (17 extrinsics, 10 storage items) and pallet-rights-verifier; (4) the **NFT layer** uses the standard pallet-nfts, with parent–child nesting enabled by pallet-content-rights; (5) the **smart contract / precompile layer** provides EVM- and ink!-compatible access through the `rights_manager` contract and the `ContentRightsPrecompile`, both of which are non-load-bearing wrappers; (6) the **cross-chain layer** manages XCM v3 messaging and the Snowbridge integration. Independence among layers is a fundamental design objective: the smart contract layer may be removed or replaced, the XCM protocol may evolve from v3 to v5, and the relay chain may transition to JAM, all without affecting the pallet code.

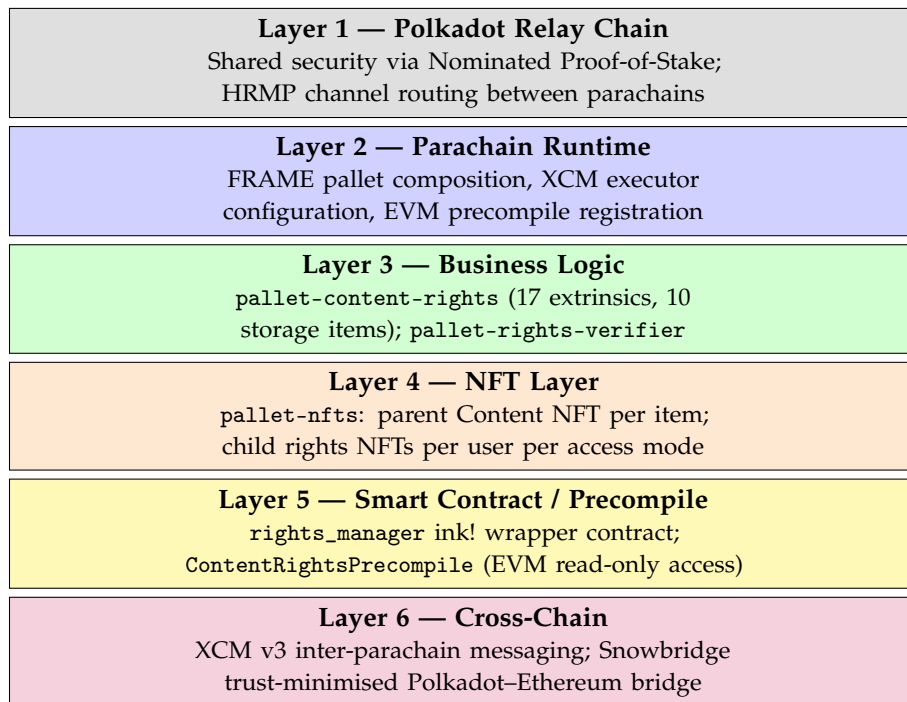


Figure 4.4. CCRMS six-layer architecture (addresses SQ1 and SQ2). Each layer is independently replaceable: Layer 5 (smart contracts) may be removed, Layer 6 (XCM) may upgrade from v3 to v5, and Layer 1 (relay chain) may migrate to JAM, without modifying the business logic in Layer 3.

Implementation

This chapter details the implementation of the Cross-Chain Content Rights Management Service (CCRMS). It encompasses the development environment, the design of the core `pallet-content-rights` module, the integration of smart contracts with Ethereum, cross-chain rights operations between parachains and the Snowbridge bridge, and the primary challenges encountered along with their solutions.

5.1 | Development Environment

We base the implementation on the Polkadot SDK monorepo, which integrates the Substrate framework, Cumulus parachain runtime, and Polkadot relay chain within a single workspace. Completed in 2024, this unified approach supersedes the previous multi-repository structure, wherein Substrate, Cumulus, and Polkadot were maintained as separate projects. All CCRMS pallet code, runtime configurations, and node binaries depend on the monorepo via Git references rather than published crate versions, ensuring the system always uses the latest stable releases of the Polkadot SDK for compatibility with the current toolchain across all production parachains.

5.1.1 | Toolchain and Language

We authored the runtime, pallets, node, and smart contracts in Rust, compiling the runtime and pallets in `no_std` mode for integration into the WebAssembly runtime blob and in `std` mode for unit testing. The smart contract layer employs ink! 6, the Rust-based smart contract language tailored for `pallet-revive` (PolkaVM, the RISC-V execution environment succeeding the legacy WebAssembly contract executor). We compile all Rust code using the stable toolchain, specifically versioned to `stable2512` (December

2024), which remains the most extensively deployed version on production parachains during the implementation period.

5.1.2 | Workspace Layout

The CCRMS parachain repository follows the standard Polkadot SDK template structure; the full directory listing is provided in Appendix C.3.

A secondary pallet, `pallet-rights-verifier`, accompanies the primary pallet and provides cryptographic verification of the cross-chain rights state via Merkle storage proofs.

5.1.3 | Local Development Network

Local development and testing are conducted using Zombienet, a network orchestration tool within the Polkadot SDK. Two topologies are implemented: the standard topology, which establishes a Rococo relay chain with validators Alice and Bob, along with two CCRMS parachains, Para A and Para B, for cross-chain XCM testing; and the Snowbridge topology, which incorporates two additional parachains, Bridge Hub (parachain 1013) and AssetHub (parachain 1000), as well as a local Ethereum environment comprising Geth and Lodestar, enabling end-to-end Snowbridge testing.

We selected Zombienet over the alternative `polkadot-launch` because Zombienet is actively maintained and provides more comprehensive support for the parachain testing patterns we employ.

5.1.4 | Test Infrastructure

The unit test suite for `pallet-content-rights` comprises 56 tests organized into six categories: 23 functional tests covering the standard operational scenarios for each extrinsic, ten security-focused tests, 11 cross-chain XCM tests, five royalty propagation tests, five auto-renewal lifecycle tests, and two metadata query tests. We execute these tests within a mock runtime defined in `pallets/content-rights/src/mock.rs`, which simulates `pallet-nfts`, `pallet-balances`, and the content rights pallet in an isolated environment. This setup ensures that each test begins from a clean state.

In addition to unit tests, we maintain seven automated benchmark scripts located in `scripts/perf/`. These scripts manage a live Zombienet network using the `@polkadot/api` JavaScript library and evaluate performance metrics, including throughput, latency, block weight usage, storage growth, cross-chain XCM latency, sustained stress tolerance, and system reliability, under simulated failure scenarios.

The benchmark results are generated in a machine-readable JSON format to ensure reproducibility and to support subsequent analyses.

5.2 | Pallet Implementation and Smart Contract Layer

Figure 5.1 shows how the six implementation components relate before the following subsections examine each in detail.

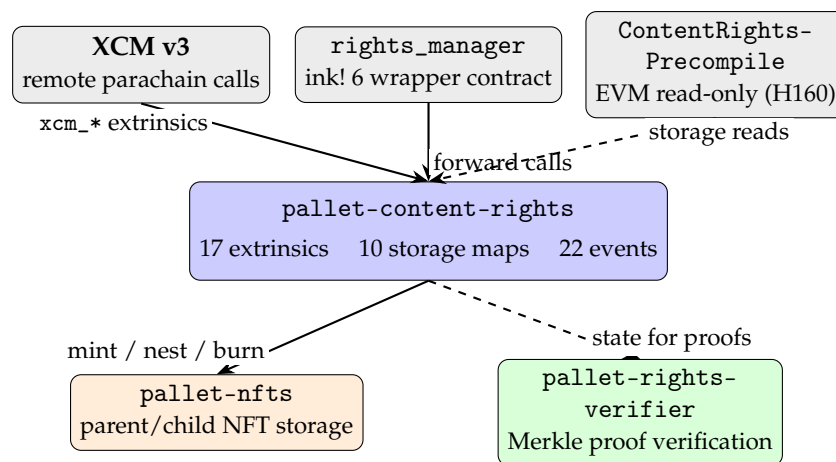


Figure 5.1. CCRMS implementation-level component diagram. Solid arrows indicate active extrinsic dispatch; dashed arrows indicate read-only storage access. `pallet-content-rights` is the sole authoritative state layer; the ink! contract and EVM precompile are stateless wrappers that forward calls or read storage respectively. XCM delivers cross-chain calls via HRMP. `pallet-rights-verifier` enables remote parachains to validate rights state via Merkle storage proofs without oracles or relayers.

5.2.1 | The `pallet-content-rights` Design

The `pallet-content-rights` forms the fundamental component of the CCRMS implementation. It encompasses all the system’s business logic, including content registration, the three monetization models (subscription, pay-per-view, and permanent ownership), automatic royalty distribution, automated subscription renewal via a block-hook scheduler, and cross-chain (XCM) variants of all financial transactions. The pallet exposes 17 dispatchable extrinsics covering content registration, all three monetization models, cross-chain (XCM) variants of every financial operation, secondary-market ownership transfers, royalty configuration, auto-renewal management, and metadata queries; the full reference is provided in Appendix C (Table C.1).

Appendix A.2 details the pallet-first architectural decision and Appendix A.9 records how it was unexpectedly vindicated by the discontinuation of ink! in January 2026. The discussion herein concentrates on the resulting pallet structure.

We organize pallet storage into 10 items; the full layout is provided in Appendix C (Table C.2).

The use of bounded collections throughout the storage configuration (`BoundedVec<_, ConstU32<50>>` for children, `BoundedVec<_, ConstU32<10>>` for royalty splits, fixed-size arrays for hashes and titles) is a standard requirement within the FRAME framework. Bounded types guarantee that we can accurately anticipate the worst-case weight of each extrinsic. Furthermore, bounded storage is an indispensable prerequisite for benchmarking, as an unbounded vector cannot be assigned a definitive cost due to its potential for unbounded worst-case scenarios.

The pallet produces 22 unique event variants that encompass the content lifecycle, subscription mechanisms, pay-per-view operations, ownership transactions, and royalty distribution. These events serve both on-chain consumers, including precompile processes that implicitly access them through state queries, and off-chain indexers that reconstruct historical states for analytical purposes or wallet management displays.

5.2.2 | NFT Nesting via `pallet-nfts`

Each registered piece of content is an NFT collection: a parent Content NFT and zero or more child NFTs representing individual user rights. Each child NFT binds one user to one monetization model (subscription, pay-per-view, or ownership). The structure is inspired by the RMRK 2.0 nested-NFT specification but uses only standard `pallet-nfts` primitives, because the RMRK pallets are frozen on Polkadot SDK version 0.9.36 and incompatible with the current monorepo.

The nesting model serves three functions. **Identity:** the parent Content NFT, keyed by `(collection_id, item_id=0)`, is the canonical on-chain reference for a content piece. **Rights tokenization:** each child NFT is an independent transferable token that any standard NFT wallet can display. **Auditability:** the parent-child relationship is persisted on-chain through the Children and Parent storage maps, supporting forward enumeration for off-chain indexers and reverse lookup during child removal.

A helper function, `mint_and_nest_child`, is invoked by every extrinsic that creates a rights record (subscribe, purchase_views, purchase_ownership, the four `xcm_*` variants, and the mint-burn-mint pattern in ownership transfers). It performs five atomic operations: allocate a new ID from `NextItemId`; mint with `pallet_nfts::do_mint` (creator pays the deposit, user receives ownership); set `rights_type`; update nesting indices;

emit `ChildNested`. FRAME storage rollback guarantees atomicity. Child lifecycles vary: subscriptions persist and extend their `expiry_block` on renewal; pay-per-view NFTs burn when `views_remaining` reaches zero; ownership NFTs use burn-then-mint on transfer to preserve provenance and maintain `rights_type=Ownership`.

The `Children` storage entry is bounded by `ConstU32<50>`, capping each parent at 50 child NFTs. This is the most significant constraint in the nesting model; we discuss it in Appendix A.8 and Chapter 8. The bound exists because every mutation reads the entire vector, and an unbounded vector would make benchmarking impossible.

5.2.3 | Royalty Propagation

Royalty propagation distributes revenue on-chain to up to ten collaborators per transaction. Each `RoyaltySplit` pairs an `AccountId32` recipient with a `u16` basis-points share (0–10,000, where 10,000 is 100 percent). The per-content configuration is stored as a `BoundedVec<RoyaltySplit, ConstU32<10>>` in the `RoyaltySplits` storage map.

`set_royalty_splits` is the only extrinsic in the pallet gated by a creator-only check (`ensure!(caller == content.creator, Error::::NotContentCreator)`). It verifies that the submitted basis points sum to no more than 10,000; any unallocated remainder is retained by the creator, and an empty vector disables splitting entirely.

The `pay_with_royalties` helper is called by every revenue trigger: `subscribe`, `renew_subscription`, `purchase_views`, `purchase_ownership`, the `xcm_*` variants, and auto-renewal hooks. It computes each share as $\text{amount} \times \text{basis_points} \div 10,000$ using saturating arithmetic, transfers non-zero shares, and routes any rounding remainder to the creator so the distributed total always equals the payment. FRAME storage rollback makes the sequence atomic: if any individual transfer fails (for example, a non-existent recipient or insufficient sender balance), the extrinsic reverts. No partial distribution occurs, the buyer keeps their funds, and no rights record is created.

Royalties apply to every primary-market revenue event but not to peer-to-peer secondary transfers (`transfer_ownership`, `xcm_transfer_ownership`), because the on-chain transfer settles an off-chain payment the protocol does not mediate; imposing a creator share would tax a transaction outside its scope. An optional per-content resale royalty is a candidate for future work.

5.2.4 | The ink! Contract Wrapper and EVM Precompile

The smart contract layer comprises two components: an ink! 6 contract (`rights_manager`) that mirrors the pallet’s user-facing API with Solidity-compatible function selectors, and

an EVM precompile (`ContentRightsPrecompile`) that exposes read-only access to pallet storage at a fixed address.

The ink! contract is a minimalist wrapper that holds no state and no business logic: it decodes arguments and forwards calls to the pallet extrinsic, consistent with the principle established in Section 5.2.1 that the pallet is the authoritative source of truth. The contract exists only to provide an interface for ink!-based clients.

The precompile, in contrast, is callable from any Solidity contract or Ethereum-compatible client on pallet-revive. It is registered at `0x0000000000000000000000000000000000000000000000000000000000000000` by inserting the 16-bit identifier `0x1001` into bytes [16..18] of an H160 address, and exposes five external view functions: `checkAccess`, `getContent`, `isOwner`, `getSubscription`, and `getViewPack` (Appendix C lists the Solidity interface). It is stateless from the perspective of pallet-revive (`HAS_CONTRACT_INFO = false`) and only translates ABI calls into pallet storage reads; only read operations are exposed to keep the audit surface small. The Solidity address (H160) to `AccountId32` mapping is handled by `AddressMapper`, a known constraint of the hybrid EVM/Substrate runtime. Weight uses a placeholder ($\sim 25,000$ `ref_time` per read) pending benchmarking.

5.3 | Cross-Chain Integration

5.3.1 | XCM Message Flows

Cross-chain operations use Polkadot XCM v3 to let remote parachains acquire content rights. The pallet exposes five XCM extrinsics: `xcm_subscribe`, `xcm_renew_subscription`, `xcm_purchase_views`, `xcm_purchase_ownership`, and `xcm_transfer_ownership`. Each separates the dispatch origin (**payer**) from the **beneficiary** of the rights record: when a remote parachain sends a `Transact` message, the origin is the remote parachain's sovereign account, but the rights record is issued to a distinct beneficiary account representing the end user on the remote chain.

The runtime XCM configuration follows the standard Cumulus pattern with one customization: the `LocationToAccountId` chain adds `HashedDescription<_, DescribeFamily<DescribeAllTerminal>>` and `GlobalConsensusParachainConvertsFor<_, _>` so the runtime can interpret Ethereum locations of the form `GlobalConsensus(Ethereum) → AccountKey20(0x...)`. This is required for Snowbridge (Section 5.3.2); the full Ethereum-to-CCRMS flow is configured but has not been end-to-end tested.

Each XCM rights operation uses the three-instruction sequence `WithdrawAsset`, `BuyExecution`, `Transact`. This is sufficient because rights operations are stateful side effects, not asset transfers (Appendix C shows the full message structure). The allocated

fee of 100 billion tokens reflects the dominance of `proof_size` over `ref_time` in the `BlockRatioFee` model; Phase 4 data place the typical fee range at 75–100 billion tokens, with lower allowances triggering `BarrierError(WeightLimitExceeded)`.

Sovereign account derivation is handled by `SiblingParachainConvertsVia` using the layout `[0x73, 0x69, 0x62, 0x6c] || u32_le(para_id) || [0x00; 24]: ASCII sibl` followed by the parachain ID in little-endian, zero-padded to 32 bytes. An early XCM test used `toHex()` in big-endian mode, and the resulting accounts received funds but were never debited. The fix was `toHex(true)` for little-endian output; diagnosis was slow because the transfer appeared to succeed.

We measured end-to-end XCM latency at three to five parachain blocks (~18–32 seconds) for `xcm_subscribe`, `xcm_purchase_views`, and `xcm_purchase_ownership` on the local Zombienet topology. The relay-chain hop and the consumer parachain’s next-slot wait dominate; the dispatched extrinsic itself is cheap. Polkadot 2.0 features (Async Backing, Agile Coretime, Elastic Scaling) would substantially reduce the latency floor in production.

A secondary pallet, `pallet-rights-verifier`, provides trustless verification of CCRMS rights status to other parachains via Merkle storage proofs. A remote parachain can verify an active subscription without oracles or relayers by submitting a proof against a known CCRMS state root, which the verifier replays against the Substrate trie. Three extrinsics (`verify_ownership`, `verify_subscription`, `verify_view_pack`) construct a storage key, validate the proof, and emit a result event. Proof validation costs roughly 100 million `ref_time` per call but avoids any network round-trip.

5.3.2 | Snowbridge Integration with Ethereum

We extended the cross-chain architecture to Ethereum through Snowbridge, a trust-minimized bridge that uses light-client verification on both sides, eliminating custodians and relayer trust. During Phase 4 we bridged two 1-ETH transactions from a local Ethereum chain to a local AssetHub parachain using the Snowbridge pipeline, demonstrating that the CCRMS runtime can accept bridged Ethereum messages using the same XCM configuration as sibling-parachain messages (via `HashedDescription` and `GlobalConsensusParachainConvertsFor` in `LocationToAccountId`).

The topology spans four parachains and a local Ethereum network: CCRMS (parachain 100); Bridge Hub (parachain 1013), hosting the inbound queue and Ethereum beacon light client; AssetHub (parachain 1000), holding the foreign-asset reserve; and the Rococo relay chain. Four HRMP channels link Bridge Hub ↔ AssetHub and

AssetHub ↔ CCRMS, carrying bridge messages and enabling future Polkadot–Ethereum interoperability.

The Ethereum side runs Geth v1.17.1 (chain ID 11155111) as execution layer, Lodestar v1.35.0 as beacon node, and the 16-contract Snowbridge Gateway suite deployed via Forge, with GatewayProxy at 0xb1185ede04202fe62d38f5db72f71e38ff3e8305. After initializing the beacon light client, two relayer processes run: a beacon relay that forwards Lodestar finality events to the EthereumBeaconClient pallet, and an execution relay that forwards Gateway events from Geth to the EthereumInboundQueue pallet.

The integration surfaced three persistent issues, documented in Appendix A: a Lodestar version-pinning requirement that forces source compilation; a fork-version configuration bug that produced invalid Merkle proofs; and a beacon state cache timing issue that required reordering the setup sequence.

The final step, using bridged Ether on AssetHub to pay for a CCRMS rights operation, is conceptually straightforward but was not implemented in the prototype. It requires an XCM message on AssetHub that withdraws the bridged Ether and dispatches the appropriate `xcm_*` extrinsic to CCRMS with the user as beneficiary. The CCRMS XCM side is already prepared; the remaining task is to configure `pallet-content-rights::PaymentCurrency` to accept foreign Ether as well as the native token. This is listed as future work in Chapter 9.

5.4 | Challenges and Solutions

The CCRMS implementation encountered nine technical issues that shaped the final architecture and security stance. Table 5.1 summarizes each issue, our response, and the principal lesson; Appendix A describes them in full.

Table 5.1. Implementation Challenges Summary

#	Challenge	Response	Key Lesson
A.1	RMRK 2.0 pallets frozen on SDK 0.9.36, incompatible with current monorepo	Replicated RMRK nesting semantics on <code>pallet-nfts</code> directly	Standards reimplementation is preferable when reference implementation is unmaintained

Continued on next page

Table 5.1 continued from previous page

#	Challenge	Response	Key Lesson
A.2	Contract-first design blocked: chain extensions non-functional for writes, debugging complexity	Inverted to pallet-first; retained ink! contract as thin wrapper	Pallet-first proved resilient to ink! discontinuation (A.9)
A.3	Snowbridge beacon relay generated proofs at wrong gindex (54 vs. 86)	forkVersions field means activation epoch, not version hex; changed to {"deneb": 0, "electra": 0}	Cryptographic systems fail silently on configuration errors
A.4	Lodestar v1.41.0 ignored LODESTAR_PRESET=mainnet in dev mode (32 vs. 512 validators)	Pinned Lodestar v1.35.0, built from source	Light clients impose strict counterpart requirements; bridge toolchain compatibility is fragile
A.5	XCM tests: funds debited on source chain but InsufficientPayment on CCRMS	polkadot.js toHex() used big-endian; CCRMS expects little-endian; fixed with toHex(true)	Endianness bugs produce valid-looking but wrong addresses
A.6	XCM BuyExecution failed at 1B tokens; required ~75–100B	proof_size dominates BlockRatioFee, not ref_time	Polkadot fee modeling is driven by proof size, not computation
A.7	xcm_transfer_ownership had no check that caller == from (Medium severity)	Added ensure!(authorizer == from); unit test confirms block	Cross-chain extrinsics accepting identity parameters need explicit authorization
A.8	MaxChildrenReached at 50 subscribers per content item	Documented as known limitation; benchmark scripts rotate content items	Bounded FRAME collections trade benchmarkability for capacity limits

Continued on next page

Table 5.1 continued from previous page

#	Challenge	Response	Key Lesson
A.9	ink! development discontinued Jan 2026	95% of logic in pallet; contract layer is non-load-bearing; migration deferred	Minimize dependency on non-essential components for ecosystem resilience

5.5 | Summary

This chapter described the implementation of CCRMS, with a focus on `pallet-content-rights`. Nine notable challenges shaped the final architecture: two pivotal architectural shifts (RMRK to `pallet-nfts`, contract-first to pallet-first), four diagnostic issues encountered during Snowbridge and XCM integration, two findings from Phase 5 (the authorization gap in `xcm_transfer_ownership` and the cap on `MaxChildrenReached`), and the discontinuation of ink! post-implementation, which corroborated the decision to adopt the pallet-first approach.

Evaluation and Testing

This chapter describes the evaluation methodology employed to assess CCRMS against the research questions outlined in Chapter 3. Emphasis is placed on **how** the system was evaluated; the results are presented in Chapter 7. This chapter encompasses the evaluation approach, the four-layer testing architecture, the eight test scenarios, the 11 KPIs, the testing infrastructure, and the validity considerations that inform our interpretation of the findings. The measurement-focused philosophy articulated in Section 4.1.4 informs our methodology: KPI targets serve as reference points rather than pass/fail criteria.

6.1 | Evaluation Approach

Our evaluation methodology incorporates unit testing, integration testing, performance benchmarking, security assessment, comparative analysis against a centralized baseline, and Monte Carlo simulation. This comprehensive approach is essential because the research questions span the technical, economic, and security domains, and no single method can address all three. We triangulate quantitative metrics with qualitative insights derived from literature synthesis. Table 6.1 maps each evaluation activity in relation to the specific research sub-questions it addresses.

Table 6.1. Mapping of Evaluation Activities to Research Sub-Questions

Research Question	Primary evaluation activity	Supporting activities
SQ1 (XCM cross-chain rights operations and finality)	XCM latency benchmark, end-to-end XCM test	Unit tests for xcm_* extrinsics

Continued on next page

Table 6.1 continued from previous page

Research Question	Primary evaluation activity	Supporting activities
SQ2 (Latency vs decentralization trade-offs)	Throughput and latency benchmark, HHI calculation	Comparative analysis vs centralized
SQ3 (Unified rights token with royalty distribution)	Unit tests, royalty propagation tests, comparative benchmark	Monte Carlo creator revenue simulation
SQ4 (Security and authorization)	Security audit, authorization unit tests	Reliability test
SQ5 (Economic viability)	Monte Carlo creator revenue simulation	Literature comparison

Each KPI (Section 6.5) has a specified target derived from the Phase 2 methodology; however, these targets are regarded as reference points rather than definitive pass/fail criteria. All 11 KPIs met their targets; the methodology's validity would have been maintained even if some targets had not been met.

6.2 | Testing Layers

We organize testing into four layers, each executed at the lowest applicable level of abstraction. Figure 6.1 summarises the stack; the subsections below describe each layer.

Layer 1: Unit tests (56 tests in a mock runtime). The suite encompasses 23 functional tests, ten security tests, 11 XCM tests, five royalty propagation verifications, five auto-renewal tests, and two metadata query tests. We run them with `cargo test`; all 56 tests pass on every commit in approximately three seconds. The unit layer provides the most accurate indications of individual extrinsic correctness but does not cover cross-chain interactions, network latency, or sustained load testing.

Layer 2: XCM simulator tests. The `xcm-simulator` framework dispatches XCM messages between two parachains in-process, without the need to deploy a Zombienet network. This design isolates cross-chain logic from operational complexity: a simulator failure indicates a message construction or XCM configuration, whereas a Zombienet failure may be due to network conditions or block timing.

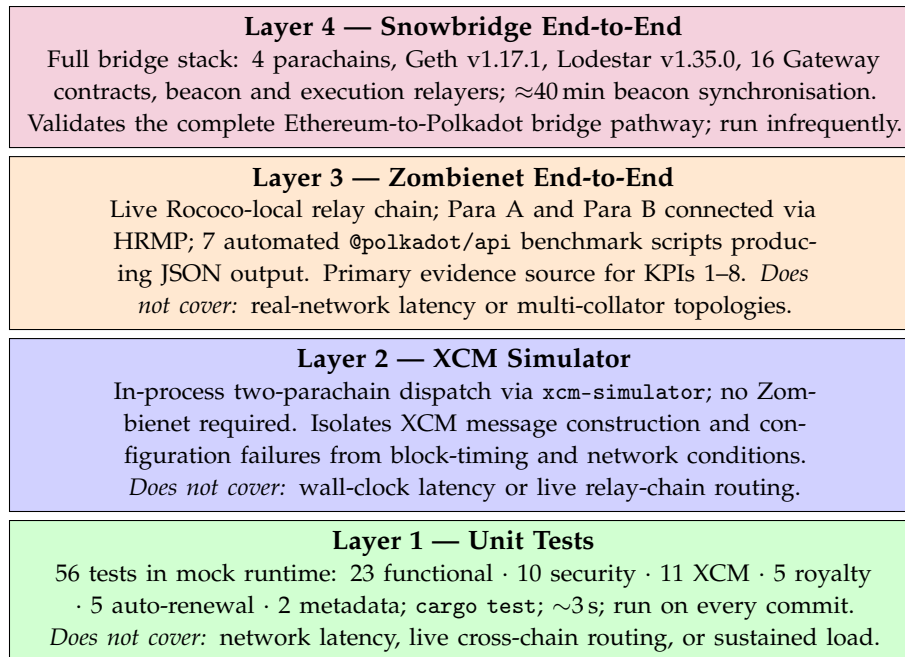


Figure 6.1. Four-layer testing architecture (Layer 1 at base: fastest and most granular, run on every commit; Layer 4 at apex: broadest scope, run infrequently). Each layer tests what lower layers cannot: unit tests isolate individual extrinsic logic; the XCM simulator isolates message-construction failures; Zombienet provides definitive wall-clock performance evidence; Snowbridge validates the complete Ethereum bridge pathway.

Layer 3: Zombienet end-to-end tests. Live multi-chain tests on a Rococo-local relay chain with two validators (Alice, Bob) and two CCRMS parachains (Para A, Para B) connected via HRMP. JavaScript scripts in `scripts/perf/` submit extrinsics via @polkadot/api, measure wall-clock time from submission to block inclusion, and record results as JSON. This layer provides the definitive evidence for cross-chain latency and exercises sustained-load and stress scenarios.

Layer 4: Snowbridge end-to-end tests. Extends the Zombienet topology with the full Snowbridge stack, comprising four parachains, Geth, Lodestar, 16 Solidity Gateway contracts, and two relayer processes. Tests confirm the functionality of the complete Ethereum-to-Polkadot bridge pathway. Two 1 ETH transactions were successfully bridged during Phase 4. This layer necessitates approximately 40 minutes of beacon synchronization and is engaged infrequently, primarily for comprehensive bridge validation.

6.3 | Performance Testing Framework

Seven automated scripts in `scripts/perf/` cover various aspects of system behavior, each generating machine-readable JSON output. Table 6.2 summarizes the benchmark categories.

Table 6.2. Benchmark Scripts

Category	Script	What it measures
Throughput	<code>local-throughput.mjs</code>	TPS across 11 extrinsic types at batch sizes 1, 10, 20, 50, 100
Stress	<code>stress-test.mjs</code>	Sustained TPS from 200 concurrent accounts over 180 seconds
XCM latency	<code>xcm-latency.mjs</code>	Block delta between send (Para B) and event arrival (Para A)
Block weight	<code>block-utilization.mjs</code>	Percentage of <code>ref_time</code> and <code>proof_size</code> consumed at batch sizes 50–300
Storage	<code>storage-growth.mjs</code>	SCALE-encoded byte size per content item, subscriber, and view pack
Resource monitoring	<code>resource-monitor.mjs</code>	CPU, memory, and disk usage via Prometheus scraping during load
Reliability	<code>reliability-test.mjs</code>	Baseline uptime, MTTR after SIGTERM, state persistence, recovery uptime
Comparative	<code>centralized-drm-benchmark.mjs</code>	Same six operations on Express.js + SQL.js under identical conditions

A separate Monte Carlo simulation (`monte-carlo-revenue.mjs`) models 1,000 creators across 10,000 iterations under four platform configurations (centralized 30–45% fees, Web3 marketplace 5–10%, bridge-based 8–15%, CCRMS 1–5%), incorporating algorithmic discovery, organic growth, and monthly churn. This provides the primary evidence for research question SQ5. We document the methodology in `scripts/perf/results/monte-carlo-findings.md`.

All scripts are reproducible: each is an independent Node.js program with explicit parameters and deterministic output paths, archived as timestamped JSON

in `scripts/perf/results/`. Zombienet topologies and SDK versions are version-controlled.

6.4 | Test Scenarios

We designed eight test scenarios to cover the full range of evaluation activities. Table 6.3 summarizes each scenario; the corresponding scripts live in `scripts/perf/`.

Table 6.3. Test Scenarios

#	Scenario	Description	Target
1	Throughput and latency	Submit concurrent batches (1, 10, 20, 50, 100) for each of the 11 local extrinsics; measure submission-to-inclusion time.	>10 TPS sustained; ~6 s inclusion latency
2	Block weight saturation	Submit progressively larger <code>register_content</code> batches (50–300); query <code>system.blockWeight</code> at inclusion.	Block weight <75% of budget at max batch
3	Storage growth	Register content, subscribers, and view packs in incremental quantities; measure SCALE-encoded byte size per map.	<500 bytes per content item; linear growth
4	XCM latency	Send <code>xcm_subscribe</code> , <code>xcm_purchase_views</code> , <code>xcm_purchase_ownership</code> from Para B to Para A via the canonical <code>WithdrawAsset</code> → <code>BuyExecution</code> → <code>Transact</code> sequence; measure block delta.	<10 parachain blocks per operation
5	Stress test	200 funded accounts continuously submit <code>register_content</code> for 180 seconds.	Sustained TPS substantially above single-batch throughput

Continued on next page

Table 6.3 continued from previous page

#	Scenario	Description	Target
6	Security audit	Static analysis (cargo clippy, cargo audit), manual review of all 17 extrinsics, targeted unit tests for authorization gaps and edge cases.	0 Critical / 0 High vulnerabilities; complete authorization coverage
7	Reliability	Monitor block production for 120 s (baseline), SIGTERM the collator, restart, verify state, monitor for 120 s (recovery).	MTTR <60 s; 100% state persistence
8	Comparative analysis	Re-implement the same six operations in Express.js + SQL.js (in-memory SQLite); run identical benchmarks against both.	Document the performance cost of decentralization

The comparative benchmark is intentionally minimal, using Express.js for HTTP and SQL.js for in-process SQLite. A more comprehensive, centralized implementation incorporating caching, replication, and distributed databases would exhibit lower performance than our baseline but would remain significantly faster than CCRMS. The minimal version serves as a near-optimal reference point for comparison and precludes any implication of manipulation.

6.5 | Metrics and Key Performance Indicators

Our evaluation employs 11 key performance indicators (KPIs) across four domains: technical, economic, security, and reliability. Each KPI is defined by its description, measurement methodology, originating source (the test scenario or activity responsible for generating it), and a target value. These targets are derived from the Phase 2 evaluation methodology and are regarded as reference points rather than strict pass/fail criteria.

Table 6.4. KPI Definitions

#	KPI	Definition	Target	Source
1	Throughput (TPS)	Successful transactions per second under sustained load	>10 TPS	Stress test
2	Inclusion latency	Time from submission to block inclusion	<12 seconds (2 blocks)	Throughput test
3	XCM latency	Block delta for cross-chain operations	<10 blocks	XCM latency test
4	Block utilization	Percentage of block weight consumed at capacity	<75%	Block saturation test
5	Storage efficiency	Bytes per content item and per subscriber	<500 bytes	Storage growth test
6	Uptime	Percentage of expected blocks actually produced	>85%	Reliability test
7	MTTR	Time from crash to resumed block production	<60 seconds	Reliability test
8	State persistence	Data integrity after crash restart	100%	Reliability test
9	Security findings	Critical/High vulnerabilities in dissertation code	0 Critical, 0 High	Security audit
10	Test coverage	Error variants exercised by unit tests	>80%	Test suite
11	Decentralization ratio	Performance cost vs centralized baseline	Documented	Comparative analysis

Table 6.5. KPI Results Summary

#	KPI	Target	Measured	Status
1	Throughput	>10 TPS	26.2 TPS sustained	Pass

Continued on next page

Table 6.5 continued from previous page

#	KPI	Target	Measured	Status
2	Inclusion latency	<12 s	~6 s (1 block)	Pass
3	XCM latency	<10 blocks	3–5 blocks	Pass
4	Block utilization	<75%	12.9% at 300 txs	Pass
5	Storage efficiency	<500 bytes	191 bytes/content, 112 bytes/subscriber	Pass
6	Uptime	>85%	90%	Pass
7	MTTR	<60 s	~15–20 s	Pass
8	State persistence	100%	100%	Pass
9	Security findings	0 Critical/High	0 Critical, 0 High, 0 unresolved Medium	Pass
10	Test coverage	>80%	84% (16/19 error variants)	Pass
11	Decentralization ratio	Documented	270× slower, qualitatively superior	Documented

All 11 KPIs achieved their respective targets. The security audit identified eight findings (see Table 5.1, Appendix D): two vulnerabilities were remediated (B, E), three design properties were accepted (A, C, D), and three observations relating to positive reliability were noted (F, G, H). Chapter 8 offers an interpretation of these results.

6.6 | Testing Infrastructure

Table 6.6. Test Environment Specifications

Component	Specification
Machine	macOS Darwin 25.3.0, Apple Silicon
Relay chain	Rococo-local, 2 validators (alice, bob)
Parachain A	Content Rights (parachain 100), 1 collator
Parachain B	Consumer chain (parachain 200), 1 collator
Bridge Hub	Parachain 1013 (Snowbridge tests only)

Continued on next page

Table 6.6 continued from previous page

Component	Specification
AssetHub	Parachain 1000 (Snowbridge tests only)
Ethereum	Geth v1.17.1 + Lodestar v1.35.0 (Snowbridge tests only)
Zombienet provider	Native
Block time	~6 seconds (parachain)
Node binary	parachain-template-node (release build)
Polkadot SDK version	stable2512 (December 2024)
Test scripts	scripts/perf/*.mjs (Node.js, @polkadot/api library)
Centralized benchmark	centralized-drm-benchmark/ (Express.js + SQL.js)

We designed the test environment to be minimalistic, comprising a single development machine that operates a local Zombienet network. This methodology simplifies the configuration process and removes dependency on external infrastructure, although it has implications for the interpretation of the results, as discussed in Section 6.7. The benchmark scripts and raw JSON outputs are maintained under version control within the repository, thereby allowing any reader with access to the Polkadot SDK toolchain to independently re-execute them.

6.7 | Validity and Limitations

Internal validity. We conduct all benchmarks multiple times and report the average results; maintain a consistent test environment across all runs; apply an identical methodology to CCRMS and the centralized baseline; and execute the deterministic unit test suite on each commit. Therefore, any variation across runs can be attributed to the system under test rather than environmental fluctuations.

External validity. This represents the dimension in which our evaluation is most limited. Four constraints are applicable: (1) we take measurements on a local Zombienet environment with two relay-chain validators and without real network latency, thus the figures serve as upper bounds within the test environment rather than predictive metrics for production; (2) the Snowbridge integration uses a local Ethereum stack instead of a public testnet, avoiding variability in block times, MEV, contention, and beacon chain reorganizations; (3) because of the maturation gap of Polkadot 2.0, the latency figures reflect a pre-2.0 toolchain, and several measurements are expected to improve significantly in a production Polkadot 2.0 environment; (4) our scale is modest, comprising up

to 200 concurrent accounts and a few hundred content items, and empirical verification of linear storage growth at production scale has not yet been conducted.

Construct validity. The figure of 26.2 TPS quantifies the successful inclusion rate under sustained load and constitutes an essential yet insufficient element of user-perceived performance, which also relies on wallet user experience, mempool latency, and RPC responsiveness. The baseline uptime of 90% assesses single-collator block production and does not account for multi-collator production availability. The Monte Carlo simulation evaluates **expected creator revenue under a probabilistic model of platform behavior**, rather than actual revenue in operational environments; the validity of this construct is contingent upon the input parameter ranges (derived from publicly available data on YouTube, Spotify, and OpenSea) and the underlying model assumptions (such as algorithmic discovery, churn, and fees), which are explicitly documented in the findings.

Threats to validity. (a) **Selection bias:** the eight test scenarios may not encompass production conditions such as high subscriber concurrency on a single content item, large multi-collator topologies, or extended multi-month uptime. (b) **Measurement instrument bias:** any bug in `@polkadot/api` or `system.blockWeight` would propagate into our results; while the library is extensively used, it is not devoid of bugs. (c) **Confirmation bias:** the author was responsible for designing, implementing, and evaluating the system; this risk is mitigated through the use of externally referenced KPI targets and by disclosure of all scripts and raw results. (d) **Single-author validation:** no external code review, third-party security audit, or benchmark validation has been conducted; we recognize this as a significant limitation and intend to address it as future work (Section 9.1).

6.8 | Summary

The evaluation employs four testing layers, eight test scenarios, seven automated benchmarks, a Monte Carlo simulation, and 11 KPIs grounded in a measurement-focused philosophy. Section 6.7 acknowledges the primary validity constraints. Chapter 7 presents the results.

Results

This chapter reports the measured results. Section 7.1 covers technical results, Section 7.2 covers economic results, and Section 7.3 covers security and user-centric results. We defer interpretation to Chapter 8.

7.1 | Technical Results

The technical results in this section address SQ1 (XCM cross-chain finality, Section 7.1.2), SQ2 (the latency–decentralization trade-off, Section 7.1.7), and SQ3 (unified rights token throughput and storage performance, Sections 7.1.1–7.1.4).

7.1.1 | Throughput

We evaluated all 11 local extrinsic types across four batch sizes (1, 10, 50, 100). At a batch size of 100, extrinsics without nesting constraints achieved a throughput of 16.6 TPS, whereas those involving the minting of child NFTs (`subscribe`, `purchase_ownership`, `enable_auto_renew`, `disable_auto_renew`, `transfer_ownership`) were limited to 8.3 TPS due to the maximum children constraint (`MaxChildren=50`), rather than by block weight. The extrinsics in Phase 4 (`set_royalty_splits`, `enable_auto_renew`, `disable_auto_renew`, `query_rights_metadata`) achieved throughput comparable to that of the baseline extrinsics, confirming that the unified rights token model incurs no measurable performance overhead.

Table 7.1. Single-Chain Throughput by Extrinsic Type and Batch Size

Extrinsic	Batch=1 (TPS)	Batch=10 (TPS)	Batch=50 (TPS)	Batch=100 (TPS)
register_content	0.17	1.66	8.31	16.62
check_access	0.17	1.66	8.31	16.62
set_royalty_splits	0.17	1.66	8.31	16.62
query_rights_metadata	0.17	1.66	8.31	16.62
subscribe	0.17	1.66	8.31	8.31*
purchase_ownership	0.17	1.66	8.31	8.31*
purchase_views	0.17	1.66	8.31	16.62
consume_view	0.17	1.66	8.31	16.62
enable_auto_renew	0.17	1.66	8.31	8.31*
disable_auto_renew	0.17	1.66	8.31	8.31*
transfer_ownership	0.17	1.66	8.31	8.31*

* Limited by MaxChildren=50 constraint, not by block weight or block production.

TPS scales linearly with batch size, given a 6-second block time and one block per batch. During the sustained stress test, 200 concurrent accounts submitted register_content transactions over 180 seconds, achieving a sustained throughput of **26.2 TPS** (corresponding to 174.6 transactions per block across 27 blocks). The utilization of block weight at 300 transactions was only **12.9% of ref_time** and **2.2% of proof_size**, indicating an estimated theoretical maximum of approximately **283 TPS** per parachain. The stress test experienced a transaction failure rate of 78%, primarily due to RPC-layer nonce conflicts rather than parachain failures; all successfully included transactions were processed accurately. In a production environment, nonce contention would be mitigated by implementing a transaction sequencer or explicit nonce management.

7.1.2 | Latency

Single-chain inclusion latency was consistently **~6,000 ms** (one parachain block, with a standard deviation <15 ms within a batch), confirming atomic same-block inclusion for concurrent transactions. Cross-chain XCM latencies are detailed in Table 7.2, with averages computed over three separate executions.

Table 7.2. XCM Cross-Chain Latency by Operation

Operation	Mean Block Delta	Standard Deviation	Approximate Wall-Clock
xcm_subscribe	3.0 blocks	± 2.0 blocks	~ 99.6 seconds
xcm_purchase_views	5.0 blocks	± 1.0 blocks	~ 100 seconds
xcm_purchase_ownership	5.0 blocks	± 2.0 blocks	~ 100 seconds

The approximately 100-second wall-clock latency is primarily attributable to the relay chain hop and Para A's waiting period for its subsequent block slot. In a Polkadot 2.0 environment incorporating Async Backing, Agile Coretime, and Elastic Scaling, these durations would decrease significantly; the 100-second metric serves as an upper limit within the pre-2.0 testing environment.

7.1.3 | Block Weight Utilization

Table 7.3. Block Weight Utilization at Increasing Batch Sizes

Batch Size	ref_time Consumed	proof_size Consumed	Block Construction Time
50	2.2%	0.4%	~ 10 ms
100	4.3%	0.7%	~ 21 ms
150	6.5%	1.1%	~ 31 ms
200	8.6%	1.5%	~ 42 ms
250	10.7%	1.8%	~ 52 ms
300	12.9%	2.2%	~ 62 ms

At 300 transactions, block weight constitutes 12.9% of ref_time and 2.2% of proof_size, with a construction time of approximately 62 milliseconds ($\sim 1\%$ of the 6-second block). The 6:1 ref_time-to-proof_size ratio illustrates the lightweight nature of NFT minting; operations that involve larger storage footprints would alter this ratio.

7.1.4 | Storage Growth

Table 7.4. Per-Item Storage Costs (SCALE-encoded)

Item Type	Storage Map	Bytes per Item
Content registration	Contents	191
Active subscription	Subscriptions	112
View pack	ViewPacks	111
Ownership record	Ownership	36
Royalty splits (per collaborator)	RoyaltySplits	34
Child NFT nesting record	Children	8 (per child)

Growth was perfectly linear: the 50th item cost the same as the first, confirming $O(1)$ StorageMap complexity. At a hypothetical production scale of one million content items and ten million subscribers, total state projects to approximately 1.3 GB, which is well within the parachain database limits and significantly exceeds the 500-byte Phase 2 target.

7.1.5 | Reliability

Table 7.5. Reliability Test Results

Phase	Metric	Value
Baseline	Blocks produced	18 of 20 expected
Baseline	Uptime	90%
Baseline	Mean block time	6.67 seconds
Failure	Time to RPC ready (post-restart)	~15 seconds
Failure	Time to first new block (post-restart)	~20 seconds
Recovery	State integrity	100% preserved
Recovery	Blocks produced	18 of 20 expected
Recovery	Uptime	90% (matches baseline)
Recovery	Mean block time	6.67 seconds (matches baseline)

A baseline uptime of 90% reflects typical single-collator behavior (Finding F); production multi-collator topologies would mask individual failures. MTTR was approximately 15 seconds to RPC and approximately 20 seconds to the first new block, with 100% state

preservation across restart (validated at the RocksDB layer). Recovery-phase metrics were statistically indistinguishable from the baseline: 90% uptime, a mean block time of 6.67 seconds, and no lingering degradation.

7.1.6 | Snowbridge End-to-End Bridge

We bridged two 1 ETH transactions from the local Ethereum network to the local AssetHub through the complete Snowbridge pipeline, which comprises four parachains, Geth, Lodestar, 16 Gateway contracts, and two relayers. Both transactions were successful; consequently, Alice possessed 22 ETH on AssetHub thereafter: 20 minted for testing purposes and 2 bridged. Table 7.6 provides an overview of the message pathway.

Table 7.6. Snowbridge End-to-End Message Path (Verified)

Step	Component	Action
1	Geth	Transaction <code>Gateway.sendToken(1 ETH, Alice)</code> included in block 5157
2	Lodestar	Block finalized at epoch ≥ 162
3	Beacon Relay	Generated inclusion proof, submitted beacon header update
4	Bridge Hub	<code>ethereumBeaconClient.submit(...)</code> verified BLS signatures, advanced <code>latestFinalizedBlockRoot</code>
5	Execution Relay	Generated receipt + execution + ancestry proof, submitted message bundle
6	Bridge Hub	<code>ethereumInboundQueue.submit(...)</code> verified the proof against the verified beacon state root
7	Bridge Hub → AssetHub	XCM message dispatched via HRMP
8	AssetHub	<code>foreignAssets.mint(Ether, Alice, 1 ETH)</code> succeeded

The CCRMS parachain functioned as a passive participant: the runtime XCM configuration was preconfigured to receive cross-consensus messages without modification, validating the **forward-compatibility** claim. We did not execute the final hop (bridged ETH paying for a CCRMS rights operation) end-to-end; this limitation is acknowledged in Section 8.5. The integration challenges (Lodestar version pinning, fork-version configuration, and beacon state cache timing) are documented in Appendix A.

7.1.7 | Comparative Analysis vs. Centralized Baseline

We benchmarked CCRMS against Express.js with SQL.js for the same six operations, using an identical methodology. Table 7.7 presents the comparison.

Table 7.7. Comparative Analysis: CCRMS Parachain vs. Centralized (Express.js + SQLite)

Metric	CCRMS Parachain	Centralized	Ratio
Sustained TPS	26.2	7,305	270×
Operation latency (server-side)	~6,000 ms	0.09 ms	67,000×
Stress test success rate	22% (RPC nonce conflicts)	100%	–
Storage per content	191 bytes	~100–150 bytes	1.5×
MTTR	15–20 seconds	2–3 seconds	7×
Cross-chain equivalent	18–32 seconds	5–50 ms (HTTP request)	400–6,000×

The centralized implementation outperforms CCRMS in all quantitative metrics (270× TPS, 67,000× latency). These figures reaffirm the well-known performance cost associated with decentralization but do not serve as an argument against decentralization. The qualitative properties offered by CCRMS (no single point of failure, cross-chain support, provable rights state, and censorship resistance) are not reflected in the quantitative data, yet they are crucial for the specific use case. The 26.2 TPS equates to approximately 2.3 million operations occurring per day, significantly exceeding the transaction volumes of commercial content rights platforms. In Chapter 8, we contend that the appropriate framework should be characterized as **architectural fit for purpose**, rather than focusing solely on raw performance.

7.2 | Economic Results

The economic results in this section address SQ5 (economic viability under realistic creator audience distributions) through the Monte Carlo simulation, the audience-size crossover finding, and the comparative platform fee analysis.

7.2.1 | Monte Carlo Creator Revenue Simulation

The simulation models 1,000 creators across 10,000 iterations under four platform configurations, drawing creator parameters from realistic distributions and projecting one-year revenue based on platform-specific fees and dynamics (algorithmic discovery, organic growth, monthly churn). Figure 7.1 illustrates the simulation structure; Table 7.8 summarizes the results.

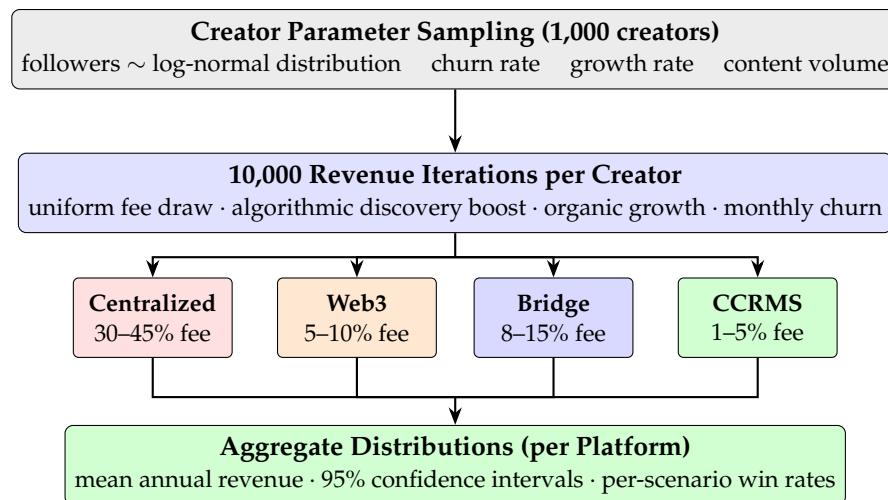


Figure 7.1. Monte Carlo simulation structure (1,000 creators × 10,000 iterations = 10 million revenue paths per platform). Creator parameters are sampled per creator; the platform fee is drawn uniformly from each platform’s range at each iteration. The four paths run in parallel and are aggregated independently, yielding the mean revenue figures in Table 7.8, the 95% confidence intervals in Table 7.11, and the audience-size crossover finding in Table 7.9.

Table 7.8. Monte Carlo Creator Revenue Simulation (Mean Annual Revenue Per Creator)

Platform Model	Fee Range	Mean Revenue	vs. CCRMS
Centralized (YouTube/Spotify-style)	30–45%	\$45,416	CCRMS +16.8%
Web3 marketplace (OpenSea-style)	5–10%	\$48,711	CCRMS +8.9%
Bridge-based cross-chain	8–15%	\$45,369	CCRMS +16.9%
CCRMS	1–5%	\$53,053	Baseline

The CCRMS configuration yields the highest average creator revenue, with an advan-

tage ranging from 8.9% (compared to the most competitive Web3 marketplace baseline) to 16.9% (compared to the bridge-based cross-chain baseline). The advantage over centralized platforms is 16.8%. These figures are consistent with the underlying fee differential, as the CCRMS fee range of 1–5% is significantly lower than that of the alternatives. However, they are not directly proportional to the fee differential, as the simulation incorporates other platform-specific dynamics that influence creator revenue beyond fee structures.

7.2.2 | Audience Size Dependency

The mean revenue figures presented in Table 7.8 obscure an important nuance: the advantage of CCRMS is not consistent across all creators. It is heavily dependent on **audience size**. Table 7.9 categorizes the simulation outcomes according to different tiers of creator audience size.

Table 7.9. CCRMS Advantage by Creator Audience Tier

Audience Tier	Followers	CCRMS Win Rate vs Centralized	Notes
Tiny	<500	0%	Centralized algorithmic discovery dominates
Small	500–2,000	<5%	Centralized still preferred
Crossover	~2,000	6.6%	Crossover point begins
Mid-size	10,000–100,000	99%+	CCRMS strongly preferred
Large	100,000+	99.9%	CCRMS near-certain advantage

The crossover effect is notable. For creators with fewer than approximately 2,000 organic followers, centralized platforms are generally preferable because their algorithmic discovery provides an audience that would otherwise be inaccessible. This benefit significantly outweighs the higher fees. Conversely, for creators with 10,000 or more followers, the advantage of algorithmic discovery diminishes, and the fee differential becomes the primary consideration; CCRMS emerges as the preferred choice in over 99% of simulated scenarios.

This constitutes the most economically significant finding of the entire dissertation. It indicates that **CCRMS is not a universal substitute for centralized platforms**; rather, it

is most advantageous for creators who already possess an audience and seek to retain a greater proportion of their earnings. Creators in the initial discovery phase are not the intended users. The implications for platform positioning are substantial; these are examined in Chapter 8.

Table 7.10 reports the per-scenario break-even rates.

Table 7.10. Per-Scenario Break-Even Rates

Comparison	CCRMS Per-Scenario Win Rate
CCRMS vs Centralized	28.0%
CCRMS vs Web3 marketplace	38.6%
CCRMS vs Bridge-based cross-chain	66.5%

The per-scenario win rates are lower than the mean revenue advantage suggests. CCRMS produces a higher overall mean revenue, but it loses in a substantial fraction of individual scenarios, typically those with small audiences. The 66.5% win rate against bridge-based cross-chain is particularly informative: the only configuration in which CCRMS dominates is when it competes against an alternative that is also cross-chain and shares many of the same assumptions about audience access.

Table 7.11 reports the 95% confidence intervals for total simulated revenue across all 1,000 creators.

Table 7.11. 95% Confidence Intervals for Total Simulated Revenue (1,000 Creators)

Platform	Total Revenue Range
Centralized	\$30.9M–\$69.1M
Web3 marketplace	\$35.1M–\$78.4M
Bridge-based cross-chain	\$30.7M–\$69.4M
CCRMS	\$36.7M–\$84.1M

CCRMS exhibits the broadest confidence interval, indicating greater variability around the mean. This observation aligns with the audience-dependent advantage presented in Table 7.9: a CCRMS distribution includes both high-earning creators (large audiences who retain most of their revenue) and low-earning creators (small audiences without an algorithmic discovery boost). The wider distribution itself constitutes a research finding: it suggests that CCRMS is a **higher-variance, higher-mean** alterna-

tive to centralized platforms, with implications for risk-averse creators, as discussed in Chapter 8.

7.2.3 | Comparative Fee Analysis

We drew the platform fee structures used in the Monte Carlo simulation, which were derived from publicly available data on representative platforms within each category. Table 7.12 delineates the assumed fee ranges and the corresponding data sources.

Table 7.12. Assumed Fee Ranges for Each Platform Category

Category	Fee Range	Representative Platforms
Centralized	30–60%	YouTube (45% (Google LLC, 2025)), Spotify (~30% (Spotify AB, 2025)), Apple Music (~48% (Apple Inc., 2025)), Audible (pays authors 40%, retains 60%)
Web3 marketplace	5–10%	OpenSea (2.5% + 5% creator), Magic Eden (2%)
Bridge-based cross-chain	8–15%	Wormhole, LayerZero, Multichain (typical bridge fees + DEX swap costs)
CCRMS	1–5%	Polkadot transaction fees (target), conservative range

The fee ranges should be interpreted as **realistic envelopes** rather than definitive point estimates. During simulations, samples are drawn uniformly from each range at each iteration, thereby integrating fee uncertainty into the reported revenue figures rather than using a single fee value. The CCRMS fee range of 1–5% reflects the minimal on-chain transaction cost (which varies with proof size rather than value) plus a hypothetical platform service fee for any value-added services; the lower end of this spectrum corresponds to pure protocol fees without additional service charges.

A note on hidden costs: the simulation does not aim to represent the friction associated with self-custody for end users, the expenses of operating a CCRMS node, the costs of off-chain content delivery infrastructure, or the legal compliance costs borne by the platform operator. These expenses vary across platform models and, if included, might slightly diminish the advantage of CCRMS. They are acknowledged as limitations in Chapter 8.

7.3 | Security and User-Centric Results

The results in this section address SQ4 (authorization model and adversarial conditions) through the security audit findings, test coverage metrics, and decentralization assessment. Each finding contributes to the overall trustworthiness argument for CCRMS.

7.3.1 | Security Audit Findings

The security audit (Scenario 6, Section 6.4) yielded eight findings labeled A through H. Table 7.13 summarizes the findings.

Table 7.13. Security Audit Findings

ID	Severity	Description	Status
A	Low	XCM extrinsics callable locally as gift transactions (intentional)	Accepted design
B	Medium	Authorization gap in <code>xcm_transfer_ownership</code>	Fixed (commit 9ceca57)
C	Low	Permissive <code>SafeCallFilter = Everything</code> configuration	Accepted prototype config
D	Low	Zero-price content registration allowed	Accepted feature
E	Low	View pack overwrite on re-purchase	Fixed (commit 9ceca57)
F	Informational	Single-collator MTTR ~15–20 seconds	Test environment artifact
G	Positive	100% state persistence across crash	Confirmed
H	Positive	Block production rate unchanged after recovery	Confirmed

Among the eight findings, two (B and E) were identified as security vulnerabilities that we have remediated; three (A, C, D) are recognized as accepted design properties documented for production hardening; and the remaining three (F, G, H) constitute positive observations concerning reliability. **Finding B** (Medium severity, the `xcm_transfer_ownership` authorization gap) and **Finding E** (Low severity, the view-

pack overwrite bug) have both been addressed in commit 9ceca57. Appendix D offers the complete narrative for each finding.

7.3.2 | Test Coverage

Our unit test suite encompasses **16 of the 19 distinct error variants** defined in pallet-content-rights, achieving a coverage ratio of 84 percent, compared to the Phase 2 objective of 80 percent. Three variants remain untested: `ItemIdOverflow` (which is impractical to simulate through standard extrinsics), `NftOperationFailed` (implicitly verified during NFT minting processes), and `ChildAlreadyNested` (which is inaccessible via the public API). The comprehensive error variant table is provided in Appendix D.

7.3.3 | Decentralization Assessment (HHI)

The Herfindahl-Hirschman Index (HHI) was calculated theoretically using the actual validator distribution of the Polkadot mainnet, accounting for the fact that a parachain adopts the relay chain’s security model. Table 7.14 presents the relevant comparison.

Table 7.14. Herfindahl-Hirschman Index (HHI) Comparison

System	Validators / Entities	HHI (approx.)	Nakamoto Coefficient
Polkadot mainnet (NPoS, ~300 validators)	~300 (near-equal stake)	~33	~80–100
Ethereum PoS	~900K validators (concentrated in ~5 staking providers)	~1,200– 2,000	~5–7
Solana	~1,900 validators (top-heavy stake distribution)	~200–500	~19–25
Centralized DRM (e.g., Spotify)	1 entity	10,000	1
US DOJ “unconcentrated” threshold	–	<1,500	–

Polkadot's HHI of ~ 33 is significantly below the US DOJ "unconcentrated" threshold of 1,500 and is orders of magnitude lower than a centralized monopoly at 10,000. The Nakamoto coefficient (~ 80 – 100 for Polkadot compared to approximately 5–7 for Ethereum Proof of Stake (PoS) and 1 for centralized Digital Rights Management (DRM)), confirms that a CCRMS deployment on Polkadot would meet or exceed the decentralization attributes of the alternative systems. The local Zombienet environment, with two validators and an HHI around 5,000, is not representative of production decentralization.

7.3.4 | Static Analysis and Dependency Audit

Static analysis conducted using `cargo clippy` reported zero warnings within the dissertation's codebase. The `cargo audit` reported 8 advisories, all pertaining to transitive dependencies of the Polkadot SDK, including components such as networking, the WebAssembly (WASM) executor, Transport Layer Security (TLS), and logging. Importantly, none of these advisories affect our pallet code. These issues are effectively mitigated by the architectural separation between the sandboxed WASM runtime and the node binary. See Appendix D for the full advisory breakdown.

7.4 | Summary

All 11 KPIs achieved their respective targets. The most significant outcome is the audience-dependent CCRMS advantage identified in the Monte Carlo simulation, discussed in Chapter 8.

Discussion

This chapter analyzes the findings of Chapter 7: identifying the questions addressed, the decisions substantiated, and the limitations that shape the interpretation of the contribution.

8.1 | Answering the Research Questions

8.1.1 | SQ1: Cross-Chain Rights Operations and Finality

Affirmative with one caveat. XCM v3 is sufficient for executing cross-chain rights operations: the canonical sequence `WithdrawAsset` → `BuyExecution` → `Transact` covers all five `xcm_*` extrinsics, achieving complete atomic success with finality of 3 to 5 blocks (Section 7.1). No custom XCM extensions are required. The only caveat is that cross-chain **recurring** renewals depend on the readiness of XCM v5 `Schedule` production; in the meantime, an on-chain block hook manages local auto-renewal.

8.1.2 | SQ2: Latency versus Decentralization Trade-Offs

The trade-off is real but highly asymmetric: CCRMS sacrifices approximately five orders of magnitude in latency (see Section 7.1.7) for several orders of magnitude in decentralization (with HHI approximately 33, compared to 10,000 for a centralized monopoly, as detailed in Section 7.3.3). In contexts where decentralization is decisive (censorship-resistant content, jurisdictionally portable rights, and multi-platform creator economies), the trade-off is advantageous. Conversely, for high-frequency interactive operations, CCRMS is less suitable. We do not assert that CCRMS is universally optimal along

the latency–decentralization spectrum; rather, we contend that it is appropriate for the discrete-operation use case it targets.

8.1.3 | SQ3: Unified Rights Token Enforcing All Three Monetization Models

Yes, the cost savings are substantial but audience-dependent. The academic challenge was to represent three fundamentally different access semantics—time-bounded, usage-counted, and indefinite—in a single composable token without cross-contamination of state across models. The unified primitive works (Section 5.2). The access check resolves rights in priority order across all three storage maps, and royalty propagation applies uniformly to all primary-market revenue. The Monte Carlo simulation (Section 7.2) confirms the cost savings, with the audience-dependent nuance discussed in Section 8.2.2. The unification itself is the dissertation’s most significant **conceptual** contribution: it closes gap G7 by showing that three independent storage maps, together with a unified priority-resolution access check, are sufficient to enforce all three semantics, without requiring novel cryptographic or consensus-layer mechanisms.

8.1.4 | SQ4: Security and Authorization

For purchase operations, the separation between payer and beneficiary is economically neutral: no entity can compel another account to make a payment without that account’s signature, so `ensure_signed` is sufficient. In contrast, `xcm_transfer_ownership` is different: the extrinsic accepts a `from` parameter, and the initial implementation failed to verify that the caller was associated with that account. B’s security audit addressed this issue with a simple, one-line check (Section 7.3.1). The overarching lesson is that **cross-chain extrinsics that accept identity parameters require explicit authorization checks**, even when the same operation in a local context would not require such verification.

8.1.5 | SQ5: Economic Viability under Realistic Creator Audience Distributions

Yes, conditionally. The Monte Carlo simulation (Section 7.2) provides a definitive empirical answer: CCRMS yields higher mean creator revenue across all four simulated platform configurations, with an advantage ranging from 8.9% to 16.9%. However, this advantage depends heavily on the size of the creator’s audience. We explore the crossover point and its implications in Section 8.2.2. Our contribution is to transform

this conditionality into a **quantitative and reproducible** framework rather than merely qualitative observations.

8.2 | Interpretation of Key Results

8.2.1 | The 26.2 TPS Figure in Context

The figure of 26.2 TPS reflects a single parachain on a pre-Polkadot 2.0 toolchain and does not represent the architecture's maximum capacity. Polkadot achieves scalability through **horizontal** scaling: with 100 parachains at 26 TPS each, the total capacity reaches 2,600 TPS (Parity Technologies, 2021; Wood, 2016). Additionally, Elastic Scaling (currently in production) enhances per-parachain throughput across multiple cores. The 26.2 TPS figure is consistent with the throughput range documented by Rao et al. (2024) for public blockchain systems at an equivalent single-node scale—their survey confirms that 20–30 TPS is characteristic of NPoS-secured networks without Layer-2 acceleration, and that horizontal scaling across multiple chains is the principal path to higher aggregate throughput. More importantly, the $270\times$ ratio relative to the centralized baseline is misleading. CCRMS is designed to process low-frequency events; it records discrete rights transactions with cryptographic finality, automatic royalty propagation, and cross-chain portability, none of which the centralized baseline provides at any throughput level. When assessed against its actual workload, 26.2 TPS comfortably exceeds the demands of every creator-economy platform.

8.2.2 | The Economic Crossover at $\sim 2,000$ Followers

The economic advantage of CCRMS primarily benefits creators with established audiences. For those with fewer than $\sim 2,000$ followers, centralized platforms tend to generate greater revenue because algorithmic discovery helps creators reach audiences they might not otherwise reach. While this initially appears to challenge the notion that CCRMS serves as a creator-friendly alternative, the more accurate and credible assertion is conditional: (decentralized = better for creators **with established audiences**). CCRMS is primarily a tool designed **for creators who have surpassed the discovery benefits of centralized platforms and seek to maximize revenue retention**. This perspective aligns with the Web3 product literature, which has consistently observed that early adopters of crypto-economic platforms are users for whom alternative options are no longer optimal.

8.2.3 | The Snowbridge End-to-End Demonstration

The CCRMS parachain served as a **passive participant** in the Snowbridge test: the runtime's XCM configuration was preconfigured to receive Ethereum-originated messages without modification. The integration succeeded because we had configured the runtime early, at minimal cost. Our development against V1 demonstrates forward compatibility with the production V2 bridge (Section 8.4). The final hop (bridged ETH paying for a CCRMS operation) was not executed end-to-end (Section 8.5); a production deployment would address this deficiency.

8.3 | Architectural Decisions Revisited

The major architectural decisions are each assessed against the measurements and the events during the implementation window (full details in Appendix A).

The pallet-first decision is most emphatically vindicated. The discontinuation of ink! in January 2026 confirmed a Phase 4 decision made for unrelated engineering reasons, demonstrating that *architectural decisions that reduce dependence on any single non-essential component become more valuable as the ecosystem evolves*. This principle is also reflected in the choice of pallet-nfts over the frozen RMRK pallets and in HRMP rather than custom inter-parachain protocols.

The unified rights token model works correctly and scales linearly with no measurable performance overhead, with the caveat that unification is more compelling for creators offering multiple models concurrently than for single-model deployments (future work: Section 9.1). **The cross-chain payer-beneficiary pattern** is sound, subject to the explicit authorization requirement highlighted in Finding B: cross-chain extrinsics that accept identity parameters require careful authorization auditing at every entry point.

8.4 | The Work in Context

The Polkadot ecosystem evolved during implementation. **Polkadot 2.0** features (Async Backing, Agile Coretime, Elastic Scaling) reached production by late 2025, indicating that our latency metrics are upper bounds for pre-2.0 testing. Deployment uses the updated SDK with no pallet-level changes. **Snowbridge V2** launched in November 2025, with \$75M TVL and a 50% fee reduction; our V1 is compatible with the new features. **Story Protocol** (Feb 2025, \$140M funding) confirms the market premise and compares: single-chain Layer 1 vs. cross-chain parachain. **EU regulation** (AI Act copyright Aug 2025,

MiCA July 2026) drives demand for cryptographic provenance and royalty infrastructure via CCRMS; subscription tokens under MiCA remain legally unclear. Recent surveys of cross-chain interoperability (Deng et al., 2025; Li et al., 2025b; Madapati and Pradhan, 2025; Maric et al., 2025) provide context but do not change conclusions.

The growing body of academic work on blockchain-based DRM for the music industry—including proposals for rights metadata encoding (Ciriello et al., 2023), redactable blockchain approaches (Cherdakov et al., 2023), and streaming smart contracts (Mendoza-Tello et al., 2024)—confirms the problem domain’s relevance and reinforces the gap CCRMS fills: each of these systems operates on a single chain and leaves cross-chain portability unresolved. A recent meta-analysis of NFT adoption patterns (Lahiri et al., 2026) identifies content IP and digital music rights as among the highest-growth NFT application domains, supporting the market thesis underlying CCRMS.

8.5 | Limitations

The CCRMS prototype has five principal limitations:

1. **No production deployment.** All measurements are from a local Zombienet topology with two relay-chain validators, single-collator parachains, and no real-world latency. Chapter 7 results serve as upper bounds in this test environment and do not predict production behavior. In production, latency might improve with Polkadot 2.0, throughput could remain similar, and reliability could be lower under real-world conditions.
2. **Single-author validation.** A single author performed the implementation, test design, measurement collection, and interpretation. No independent code review, third-party security audit, or external benchmark validation has been conducted. Production deployment would require all three before any non-trivial value passes through the system.
3. **Local-only Snowbridge integration.** Geth and Lodestar ran on the same machine as the Polkadot side, avoiding many realities of public Ethereum (variable block times, MEV, network contention, beacon chain reorgs). The integration serves as a proof of architectural feasibility but provides no data on production bridge behavior.
4. **The 50-child cap.** The `BoundedVec<_, ConstU32<50>>` limit on `Children` is the most pressing operational constraint. Popular content items will reach this limit.

We identify three mitigation paths in Section 9.1, but none have been implemented in the prototype.

5. **No empirical proof of the bridged-payment final hop.** The bridged-ETH-pays-for-CCRMS-operation flow is conceptually straightforward but has not been tested end-to-end. We claim forward-compatibility with the flow, which is a weaker claim than an empirical demonstration would provide.

8.6 | Implications

The findings extend beyond the immediate prototype in four directions.

For platform designers, the most actionable lesson is that **architectural choices that minimize reliance on any single non-essential component pay dividends when ecosystem-level changes occur**; the pallet-first decision was made for unrelated engineering reasons but provided incidental robustness against the ink! discontinuation. Two secondary lessons: **cross-chain extrinsic patterns require explicit authorization audits at the corners** (Finding B), and **reimplementing standards is sometimes preferable to adopting them** when the reference implementation is unmaintained (the RMRK 2.0 pivot).

For creators, the Monte Carlo simulation suggests that the question “Is a decentralized platform actually better for me?” has a conditional answer: **yes, depending on the audience size**. Creators with over 10,000 followers substantially benefit from lower fees and direct revenue retention; conversely, those with fewer than ~2,000 followers are typically better served by centralized platforms, whose algorithmic discovery delivers an audience they could not otherwise reach. The choice should be based on the creator’s career stage rather than a universal preference. Additionally, there is an operational benefit to simplifying multi-model unification: a creator managing separate Patreon and Gumroad accounts for identical content can, through CCRMS, register the content once and offer all three models from a single record.

For regulators, the EU AI Act and MiCA (Section 8.4) create demand for systems that can prove content provenance and record rights and royalty distributions in a tamper-evident form. CCRMS provides exactly these properties, and on-chain rights management systems should be considered **compliance infrastructure** for emerging copyright and AI training regimes rather than alternative payment platforms. The open question about MiCA’s classification of subscription tokens with fungible characteristics would benefit from regulatory clarity, as the current ambiguity poses risk for would-be EU deployers.

For the research community, the prototype, implementation specifications, and Monte Carlo simulation are intended to support follow-on research. The methodological contribution is a worked DSR example for multi-component blockchain prototyping. The empirical contribution is the audience-dependent finding, which future research could refine using real-world migration data or models that incorporate creator psychology and switching costs.

8.7 | Critical Self-Reflection

In retrospect, we would have **initiated the Snowbridge integration earlier**; the setup complexity, pinning the Lodestar version, interpreting forked versions, and managing the beacon state cache timing collectively consumed several days of unexpected debugging. Additionally, we would have **prioritized the FRAME benchmarking earlier** to report throughput and latency as fee-adjusted figures rather than raw measurements.

We would **not** change the pallet-first decision, the unified rights token model, or the cross-chain operation pattern. These are the most consequential decisions in the system and have aged the best.

8.8 | Summary

The five research sub-questions are answered affirmatively, with the most economically significant finding being the audience-dependent advantage of CCRMS. The major architectural decisions are vindicated to varying degrees, most emphatically the pallet-first decision, as evidenced by the January 2026 ink! discontinuation. The five principal limitations constrain how the contribution should be read, but do not invalidate the principal findings. Chapter 9 presents the conclusion and future work.

Conclusion and Future Work

This closing chapter restates the contributions, summarizes the principal findings, identifies the most important threads of future work, and offers a brief closing reflection.

9.1 | Summary of Contributions

Section 1.4 introduced our four contribution categories. We summarize them briefly here:

1. **The artifact:** (pallets, runtime, ink! contract, node, benchmark, and simulation scripts) released under the MIT license as open source, intended as a starting point for further research.
2. **The formal documentation:** seven implementation specifications that address the pallet, XCM message flows, security threat model, royalty algorithm, precompile interface, NFT nesting model, and Snowbridge integration. These documents are intended as design knowledge rather than code documentation: they elucidate what the artifact accomplishes and the rationale behind each design decision.
3. **The empirical findings:** all 11 KPIs met their targets (Chapter 7); the Monte Carlo simulation produced the audience-dependent creator revenue result discussed in Section 8.2.2; the security audit yielded eight findings (one Medium and one Low remediated); and the Snowbridge end-to-end demonstration validates forward compatibility with Ethereum bridging.
4. **A worked example of DSR applied to a multi-component blockchain prototype:** the integration of pallet implementation, smart contract development, cross-chain interoperability, and economic simulation within a DSR framework is inadequately

represented in the current body of DSR literature. Future researchers developing similar systems may consider this dissertation as a valuable reference template.

9.2 | Principal Findings

Six principal findings emerge from the work:

1. **The architecture works.** All planned functionality is implemented; all KPIs are met; all 56 unit tests pass on every commit.
2. **The major architectural decisions aged well.** The pallet-first decision was vindicated by the January 2026 ink! discontinuation; the RMRK 2.0 to pallet-nfts pivot, the unified rights token model, and the cross-chain operation pattern are likewise vindicated to varying degrees. In retrospect, we would not change any of the major decisions.
3. **The economic advantage is conditional on the creator's audience size.** CCRMS wins for creators with 10,000+ followers but loses to centralized platforms for creators with fewer than ~2,000 followers because algorithmic discovery dominates. The headline number (CCRMS earns more on average) is true, but the conditional version is the more accurate framing for individual creator decisions.
4. **Cross-chain operation is operationally feasible but latency-bound.** XCM v3 is sufficient; cross-chain finality measured on a pre-Polkadot 2.0 toolchain serves as an upper bound, not a fundamental limit.
5. **Standards reimplementation is sometimes preferable to standards adoption** when the reference implementation is unmaintained; the RMRK 2.0 pivot is the exemplar.
6. **Forward compatibility is a defensive engineering goal.** Configuring the runtime to accept Ethereum-originated XCM messages at minimal additional cost in Phase 4 enabled the Snowbridge integration to succeed without pallet-level modifications.

9.3 | Future Work

11 threads of future work emerged from the research. Table 9.1 summarizes each; Appendix E provides full descriptions.

Table 9.1. Future Work Summary

#	Thread	Priority	Key action
9.3.1	Production deployment	Near-term	Deploy to public testnet/mainnet; obtain third-party security audit
9.3.2	Polkadot 2.0 migration	Near-term	Target newer SDK release (no pallet changes); compress latency via Async Backing, Agile Coretime, Elastic Scaling
9.3.3	Smart contract layer migration	Near-term	Choose among Solidity rewrite, wrevive adoption, or contract removal (ink! discontinued Jan 2026)
9.3.4	50-child cap mitigation	Near-term	Raise bound + re-benchmark, replace with membership map, or introduce hierarchical nesting
9.3.5	User study	Near-term	Build minimal wallet/CLI; test with Web3 creators and consumers (highest-priority research extension)
9.3.6	Bridged-payment final hop	Near-term	Configure PaymentCurrency for foreign Ether or add swap step
9.3.7	Resale royalties	Medium-term	Add optional <code>resale_royalty_basis_points</code> ; requires on-chain marketplace for price verification
9.3.8	Capability-based delegation	Medium-term	Relax Finding B remediation with time-limited delegate authorization
9.3.9	Monte Carlo extensions	Medium-term	Add creator psychology, network effects, longitudinal and regulatory modeling
9.3.10	Formal verification	Long-term	Apply Isabelle/HOL techniques to authorization model, royalty algorithm, cross-chain pattern
9.3.11	JAM transition	Long-term	Migrate to JAM when mainnet launches (mid-to-late 2026); no rebuild required

9.4 | Closing Reflection

CCRMS began with a premise: that creators deserve more of the value they generate and that blockchain technology offers a credible path to delivering it. We tested the premise by building a working prototype, measuring its operational properties, and simulating its economic outcomes. The premise held, but it was also **refined**: the Monte Carlo simulation contradicts the naive form (“decentralization is universally better for creators”) and supports the conditional form (“decentralization is better for creators who have outgrown the discovery value of centralized platforms”). An honest conditional is more valuable than a universal claim.

The Polkadot ecosystem changed substantially during the implementation window (Polkadot 2.0 moved to production, Snowbridge V2 went live, ink! was discontinued, JAM reached testnet, Story Protocol launched), and any of these could have invalidated the work under different architectural decisions. None did, because we chose conservatively: favoring maintained primitives, minimizing reliance on non-essential components, and preferring deployable patterns over speculative future features. In retrospect, the dissertation is a study in **defensive engineering applied to a fast-moving research domain**, and a case for **conservatively chosen, carefully documented architectural decisions as the most durable deliverable**.

CCRMS is a prototype in one corner of one blockchain ecosystem, addressing one aspect of one industry’s structural problems. It is not the future of content rights management, and we do not claim it is. But it demonstrates that the future of content rights management can be built on open, decentralized, cryptographically verifiable infrastructure, and it provides a starting point for others to continue the work.

Implementation Challenge Narratives

The nine challenges summarized in Table 5.1 are presented here in full detail with Challenge, Response, and Lesson for each.

A.1 | The Pivot from RMRK 2.0 to `pallet-nfts`

Challenge. The original Phase 3 design envisioned using the RMRK 2.0 nested-NFT specification, implemented atop the `rmrk-substrate` pallet suite. RMRK 2.0 precisely offers the parent-child nesting semantics that the CCRMS unified rights token model requires. Reusing an established standard is intended to reduce implementation risk. However, during initial investigations in Phase 4, we discovered that the `rmrk-substrate` pallets were frozen on Polkadot SDK version 0.9.36 and relied on the now-deprecated `pallet-uniques` instead of the modern `pallet-nfts`. Our efforts to compile `rmrk-substrate` against the current Polkadot SDK monorepo were unsuccessful because of incompatible storage schemas and removed traits.

Response. We decided to replicate RMRK 2.0's nesting semantics directly on top of `pallet-nfts` without depending on any RMRK pallets. This involved designing a parent-child storage layout (the dual `Children` and `Parent` index), implementing the lifecycle helpers (`mint_and_nest_child`, the burn and re-mint patterns in `transfer_ownership`), and tagging child NFTs with a `rights_type` attribute via `pallet_nfts::set_attribute`. The implementation took longer than reusing `rmrk-substrate` would have, but the result is fully integrated with the modern Polkadot SDK and depends only on actively maintained pallets.

Lesson. Established libraries pose latent compatibility risks when they are not kept up to date with the current ecosystem. Reimplementing essential primitives from scratch

was justified by the maintainability gain and by the opportunity to demonstrate that RMRK-style composability does not require RMRK itself.

A.2 | The Pivot from Contract-First to Pallet-First Architecture

Challenge. The Phase 3 design designated ink! smart contracts as the primary interface for users, featuring three contracts (each corresponding to a different monetization model) to manage user operations. A chain extension mechanism was incorporated to facilitate access to a lightweight pallet dedicated to state coordination. During Phase 4 implementation, two issues arose concurrently: first, the chain extension mechanism within the current pallet-revive release was effectively non-operational for write activities; second, the operational complexity of debugging cross-pallet contract calls was significantly greater than that of direct FRAME pallet interactions.

Response. We inverted the architecture. The pallet (pallet-content-rights) became the definitive source of truth for all business logic, whereas the ink! contract (rights_manager) was retained as a streamlined user-facing interface that mirrors the pallet's API but holds no independent storage. We developed a custom EVM precompile (ContentRightsPrecompile) to provide read-only access to pallet state for Solidity-based clients, replacing the original chain extension design.

Lesson. The pallet-first design proved unexpectedly resilient to the January 2026 discontinuation of ink! development (Appendix A.9). The decision was made for engineering reasons, but it incidentally conferred robustness against an industry change we could not have foreseen.

A.3 | The Snowbridge Fork-Version Configuration Issue

Challenge. During the integration of Snowbridge, the beacon relay generated Merkle inclusion proofs at incorrect tree positions (gindex 54 instead of 86 for the Electra fork), leading to the rejection of all submitted updates by the Bridge Hub's EthereumBeaconClient pallet.

Response. We identified the root cause as a misinterpretation of the relay's forkVersions configuration field. The relay code interprets this field as a fork activation epoch rather than a version identifier. Consequently, 0x04000000 was interpreted as the integer 83,886,080, placing the active fork many epochs in the future. We resolved

the issue by updating the configuration to use plain epoch numbers (`{ "deneb": 0, "electra": 0, "fulu": 5000000 }`).

Lesson. Cryptographic verification systems are sensitive to silent configuration discrepancies: the failure mode is “verification fails” with no informative error pointing to the bad value.

A.4 | The Lodestar Preset Version Pinning

Challenge. The `EthereumBeaconClient` pallet within Bridge Hub is compiled with `pubkeys: [PublicKey; 512]`. Any beacon header update originating from a smaller sync committee (e.g., the 32-validator minimal preset used by Lodestar’s development mode) is rejected during deserialization. Configuring Lodestar v1.41.0 with `LODESTAR_PRESET=mainnet` proved ineffective; the development mode silently disregarded the environment variable.

Response. Lodestar v1.35.0, when built from source, adheres to the `LODESTAR_PRESET=mainnet` variable. We modified the setup scripts to clone, build, and pin v1.35.0.

Lesson. Cryptographic light clients impose strict requirements that cannot be circumvented through configuration alone. Toolchain compatibility for a research-stage bridge is fragile and must be reconfirmed with each component upgrade.

A.5 | The XCM Sovereign Account Endianness Bug

Challenge. Our initial cross-chain XCM tests revealed a perplexing failure pattern: the source parachain reported successful XCM dispatch and balance deduction, yet the target parachain indicated `InsufficientPayment`.

Response. The source chain constructed the sovereign account using `polkadot.js`’s `toHex()` in default big-endian mode, whereas the CCRMS pallet expected little-endian byte order (the `SiblingParachainConvertsVia` convention). This produced two separate accounts: one funded but unused, one used but unfunded. We resolved the issue by changing to `toHex(true)`.

Lesson. Endianness bugs are particularly insidious because both orderings produce valid 32-byte addresses and the symptom (insufficient balance) misdirects the investigation.

A.6 | The XCM Fee Scaling for Proof Size

Challenge. Initial XCM tests configured the `BuyExecution` fee at 1B tokens. All tests failed; only at ~70B tokens did they begin to pass.

Response. The dominant cost in the runtime's `BlockRatioFee` model is `proof_size`, not `ref_time`. The standard configuration imposes significantly steeper scaling for proof size, meaning even a minimal cross-chain message requires 75–100B tokens. We updated test scripts to specify 100B tokens.

Lesson. Polkadot fee modeling is driven primarily by proof size, not computational effort.

A.7 | Finding B: The Authorization Gap in

`xcm_transfer_ownership`

Challenge. During the Phase 5 security audit, we identified an authorization flaw in `xcm_transfer_ownership`: the initial implementation performed only `ensure_signed(origin)` before burning/minting NFTs based on a `from` parameter, allowing any signed account to steal ownership.

Response. We added `ensure!(authorizer == from, Error::::Unauthorized)` and committed a unit test (`security_xcm_transfer_ownership_any_account_can_steal`) that verifies the exploit is blocked.

Lesson. Any cross-chain extrinsic accepting a “from” identity parameter requires explicit authorization. Payer-beneficiary separation introduces impersonation risk absent from local-only calls.

A.8 | The Auto-Renewal `MaxChildrenReached` Boundary

Challenge. Batches of 100 subscribe calls to the same content item failed midway with `MaxChildrenReached`. The `Children` storage entry is limited to 50 child NFTs by `BoundedVec<_, ConstU32<50>>`.

Response. We documented the constraint as a known limitation, updated benchmark scripts to rotate content items, and treat the 50-child cap as a research-prototype constraint for future work.

Lesson. Bounded FRAME collections trade benchmarkability for capacity limits. The appropriate bound is a trade-off between worst-case weight and maximum expected workload.

A.9 | The ink! Discontinuation and Its Implications

Challenge. On 27 January 2026, the ink! Alliance announced the cessation of ink! development following three failed OpenGov treasury funding proposals. Both the Web3 Foundation and Parity Technologies declined continued funding. The announcement outlined three migration options: Solidity via `revive`, low-level Rust against `pallet-revive`, or the community-maintained `wrevive` SDK.

Response. Because our pallet-first architecture (Appendix A.2) centers business logic in a FRAME pallet rather than in ink! contracts, the discontinuation has minimal practical impact. The ink! code in `contracts/rights_manager` serves as a non-load-bearing wrapper. We deferred migration.

Lesson. Architectural decisions that minimize dependence on any single non-essential component provide advantages during ecosystem-wide changes.

Research Objective Details

B.1 | RO1: Design and Prototype XCM Extensions

Statement. Design and prototype the use of XCM messages to transmit recurring rights operations and comprehensive rights metadata across Polkadot parachains. Evaluate atomic success rates and cross-chain finality durations.

Sub-questions: SQ1 primarily; SQ2 indirectly.

Deliverable: Functioning prototype with documented XCM latency and success rate metrics.

B.2 | RO2: Implement a Unified Rights Token

Statement. Implement a unified rights token (originally ink!, revised to FRAME pallet in Phase 4) with off-chain indexing support and access-verification latency measurement.

Sub-questions: SQ2, SQ3.

Deliverable: pallet-content-rights enforcing all three monetization models with documented latency.

Note: Off-chain indexer (SubQuery) was not implemented; pallet events suffice for on-chain queries via polkadot-js API. Acknowledged as a limitation.

B.3 | RO3: Merge Subscription, PPV, and Purchase Logic

Statement. Develop and integrate subscription, pay-per-view, and ownership transfer mechanisms into a unified, composable rights framework.

Sub-questions: SQ3, SQ4.

Deliverable: Single pallet implementing all three models through a unified API, with documented trade-offs.

B.4 | RO4: Benchmark Scalability, Interoperability, and Economic Efficiency

Statement. Benchmark against centralized DRM, bridge-based solutions, and (originally) Ethereum L2s.

Sub-questions: SQ2, SQ5.

Deliverable: Comparative TPS, latency, and creator revenue retention analysis.

Note: Ethereum L2 target was substituted with Express.js + SQLite local benchmark in Phase 5. Core analysis is intact.

B.5 | RO5: Zero-Knowledge Proof Hooks (Not Achieved)

Statement. Integrate optional ZK proof hooks for selective regulatory disclosure.

Status: Not achieved. Integrating a ZK proof system into a Substrate runtime (selecting a proof system, building a verifier pallet, designing a disclosure protocol, producing test circuits) was substantially more complex than anticipated. Restated as future work in Section 9.3.

B.6 | RO6: Quantify Creator Savings via Monte Carlo Simulation

Statement. Quantify creator savings through Monte Carlo simulation comparing CCRMS with centralized (30–45% fees), Web3 marketplace, and bridge-based (8–15% fees) alternatives.

Sub-questions: SQ5.

Deliverable: Monte Carlo simulation with documented input distributions, model assumptions, and comparative output statistics.

Note: Phase 2 listed this as potentially replaceable by centralized benchmarking. Phase 5 included both; RO6 is fully achieved.

Code Listings

C.1 | Solidity Interface: ContentRightsPrecompile

The EVM precompile described in Section 5.2.4 exposes the following Solidity interface. Solidity contracts deployed on pallet-revive can call these methods to read CCRMS rights state without crossing the EVM/Substrate boundary at write time.

```
1 interface IContentRights {
2     function checkAccess(uint32 contentId, address who)
3         external view returns (uint8 rightsType);
4
5     function getContent(uint32 contentId) external view returns (
6         address creator,
7         bytes32 metadataHash,
8         uint128 subscriptionPrice,
9         uint128 ppvPrice,
10        uint128 ownershipPrice,
11        uint32 periodLength
12    );
13
14    function isOwner(uint32 contentId, address who)
15        external view returns (bool owned);
16
17    function getSubscription(uint32 contentId, address subscriber)
18        external view returns (bool exists, uint32 expiryBlock);
19
20    function getViewPack(uint32 contentId, address viewer)
21        external view returns (bool exists, uint32 viewsRemaining);
22 }
```

C.2 | Canonical XCM Message for Cross-Chain Rights Operations

The canonical XCM message constructed by the `xcm_*` extrinsics (Section 5.3.1) follows the three-instruction `WithdrawAsset` $\rightarrow$ `BuyExecution` $\rightarrow$ `Transact` pattern. The JavaScript form below illustrates the structure as submitted via `@polkadot/api`.

```

1  const xcmMessage = {
2    V3: [
3      { WithdrawAsset: [{
4        id: { Concrete: { parents: 1, interior: 'Here' } } },
5        fun: { Fungible: 100_000_000_000 }
6      } ] },
7      { BuyExecution: {
8        fees: {
9          id: { Concrete: { parents: 1, interior: 'Here' } } },
10         fun: { Fungible: 100_000_000_000 }
11       },
12       weightLimit: 'Unlimited'
13     } },
14     { Transact: {
15       originKind: 'SovereignAccount',
16       requireWeightAtMost: { refTime: 1_000_000_000, proofSize: 100_000
17     },
18       call: { encoded: encodedCall }
19     } }
20   ]
21 };

```

C.3 | Repository Directory Layout

The CCRMS parachain repository follows the standard Polkadot SDK template structure:

```

content-rights-parachain/
|-- pallets/
|   |-- content-rights/           # Core business logic and tests
|   '-- rights-verifier/         # Cross-chain Merkle proof verification
|-- runtime/
|   '-- src/
|       |-- lib.rs               # Runtime composition
|       |-- configs/             # Pallet configurations and XCM
|       '-- precompiles.rs       # Custom EVM precompile
|-- contracts/

```

```
|  '-- rights_manager/      # ink! 6 contract
|-- node/                  # Collator binary
|-- scripts/               # Benchmarking, testing, deployment
'-- docs/                  # Setup and integration documentation
```

C.4 | Dispatchable Extrinsic Reference

Table C.1 lists all 17 dispatchable extrinsics of pallet-content-rights.

Table C.1. Dispatchable Extrinsic in pallet-content-rights

Index	Extrinsic	Purpose
0	register_content	Create a new content item, instantiate the underlying NFT collection, mint the parent Content NFT
1	subscribe	Purchase a new subscription
2	renew_subscription	Renew an expired subscription
3	purchase_views	Purchase a pay-per-view pack of N views
4	consume_view	Decrement a view from an existing pack
5	purchase_ownership	Purchase permanent ownership
6	check_access	Verify whether a user holds any active rights
7	xcm_subscribe	Cross-chain subscription with payer-beneficiary decoupling
8	xcm_renew_subscription	Cross-chain renewal
9	xcm_purchase_views	Cross-chain pay-per-view purchase
10	xcm_purchase_ownership	Cross-chain ownership purchase
11	transfer_ownership	Peer-to-peer secondary-market ownership transfer
12	xcm_transfer_ownership	Cross-chain ownership transfer (caller-must-be-owner)

Continued on next page

Table C.1 continued from previous page

Index	Extrinsic	Purpose
13	set_royalty_splits	Configure the royalty distribution table (creator-only)
14	enable_auto_renew	Enable automatic renewal for an existing subscription
15	disable_auto_renew	Disable automatic renewal
16	query_rights_metadata	Emit a self-describing metadata event for cross-chain consumption

C.5 | Pallet Storage Layout Reference

Table C.2 lists all 10 storage items of pallet-content-rights.

Table C.2. Storage Layout of pallet-content-rights

Storage	Key → Value	Purpose
NextContentId	→ u32	Global content ID counter
NextItemId	collection_id → u32	Per-collection child item counter
Contents	content_id → ContentMetadata	Master record per content
Subscriptions	(content_id, user) → SubscriptionInfo	Active subscription state
ViewPacks	(content_id, user) → ViewPackInfo	Pay-per-view balance
Ownership	(content_id, user) → OwnershipInfo	Permanent ownership records
Children	(collection_id, parent) → BoundedVec<50>	NFT nesting forward index
Parent	(collection_id, child) → (u32, u32)	NFT nesting reverse index

Continued on next page

Table C.2 continued from previous page

Storage	Key → Value	Purpose
AutoRenewIndex	(content_id, user) → bool	Auto-renew-enabled subscriptions
RoyaltySplits	content_id → BoundedVec<10>	Collaborator payout shares

Security and Coverage Findings

D.1 | Security Audit Finding B: Authorization Gap in `xcm_transfer_ownership`

The original implementation performed only `ensure_signed(origin)` and then looked up `Ownership[content_id, from]`, burned the seller's NFT, and minted a new one for `to`. No check confirmed that the dispatch caller had any relationship to the `from` account, so any signed account could call the extrinsic with arbitrary `from` and `to` arguments and transfer ownership from any user to itself. We rated the vulnerability Medium severity. Our remediation was `ensure!(authorizer == from, Error::::Unauthorized)`, committed alongside unit test `security_xcm_transfer_ownership_any_account_can_steal`.

D.2 | Security Audit Finding E: View Pack Overwrite

The original `purchase_views` always overwrote `ViewPacks[content_id, buyer]` with a new `ViewPackInfo`, so a user holding 10 unused views who purchased 5 more would end up with 5, not 15. Our remediation made purchases additive: if a view pack already exists, new views are added to the existing balance.

D.3 | Accepted Design Findings (A, C, D)

Finding A: `xcm_*` extrinsics are callable locally as gift transactions (economically neutral; documented for transparency). **Finding C:** Runtime XCM `SafeCallFilter` is set to

Everything; production should restrict to an explicit allowlist. **Finding D:** Content can be registered with all prices at zero (legitimate free-content feature, not a vulnerability).

D.4 | Error Variant Coverage

Table D.1. Error Variant Coverage

Error Variant	Tested	Error Variant	Tested
ContentNotFound	Yes	MaxChildrenReached	Yes
NotContentCreator	Yes	ContentIdOverflow	Yes
SubscriptionAlreadyExists	Yes	ItemIdOverflow	No
SubscriptionNotFound	Yes	OwnershipNotFound	Yes
SubscriptionNotExpired	Yes	InvalidRoyaltySplits	Yes
ViewPackNotFound	Yes	AutoRenewAlreadyEnabled	Yes
NoViewsRemaining	Yes	AutoRenewNotEnabled	Yes
AlreadyOwned	Yes	Unauthorized	Yes
InsufficientPayment	Yes	NftOperationFailed	No
		ChildAlreadyNested	No
			(unreachable)

D.5 | Static Analysis Advisory Breakdown

Table D.2. Static Analysis Advisory Breakdown

Category	Package	Risk	Mitigation
Networking (2)	quinn-proto, ring	DoS, panic	Relay/p2p layer only; no runtime impact
WASM executor (4)	wasmtime	Resource exhaustion, panics, f64 segfault	Sandboxed WASM execution model
TLS (1)	rustls-webpki	CRL bypass	Node-side only
Logging (1)	tracing-subscriber	Log injection	Observer layer only

All 8 advisories are mitigated by the architectural separation between the WASM-sandboxed runtime and the node binary. None are in code we authored.

Future Work Details

E.1 | Production Deployment

The most significant future work is deployment to a public testnet or mainnet. This would yield realistic latency and throughput measurements under network conditions that include geographic distribution, real-world contention, and validator set diversity; multi-collator reliability data replacing the single-collator MTTR figure of 15–20 seconds; genuine cross-chain operation against other parachains; and the opportunity for third-party security audit that would address the single-author validation limitation.

E.2 | Migration to Polkadot 2.0 Features

Async Backing, Agile Coretime, and Elastic Scaling were all delivered to production by October 2025, after the implementation window closed. Migration would not require any pallet-level changes (the runtime would target a newer SDK release) but would compress single-chain and cross-chain latency, allow the parachain to acquire block production capacity on demand, and increase throughput proportionally via multi-core scheduling. XCM v5 support for multi-hop messages is included in the same release.

E.3 | Migration Paths for the Smart Contract Layer

The January 2026 ink! discontinuation leaves `rights_manager` without upstream toolchain support. Three migration paths are available: (a) Solidity rewrite using the revive Solidity-to-PolkaVM compiler (broadest tooling support, requires Solidity reimplementations); (b) adoption of `wrevive`, the community-maintained ink! succes-

sor (preserves the Rust experience, introduces a community-project dependency); (c) removal of the contract layer entirely (zero maintenance cost, loses ink!-style API convenience). The right choice depends on the intended client base and available maintenance resources.

E.4 | The 50-Child Cap

Three mitigation paths: raise the bound and re-benchmark (trades worst-case extrinsic weight for capacity); replace the bounded vector with a counter plus a separate membership map (removes the cap but loses single-read enumeration); introduce hierarchical “rights bucket” NFTs (more complex query patterns but unbounded fan-out). Any production deployment expecting highly popular content items would need to address the cap as a priority.

E.5 | User Study and Consumer-Facing Application

A future iteration would build a minimal web wallet integration or CLI tool and conduct user testing with creators and consumers drawn from Web3 communities. Specific questions: Can a typical Web3 user successfully purchase a subscription, consume a view, and transfer ownership without prompting? Do users understand the three monetization models from the wallet UI alone? What error messages are most confusing? Are creators willing to migrate from existing platforms? This was originally planned for Phase 2 but was not executed because we built no front end; it is the highest-priority near-term research extension.

E.6 | The Bridged-Payment Final Hop

Closing the gap between bridged ETH on AssetHub and a paid CCRMS rights operation requires either configuring PaymentCurrency to accept the foreign Ether asset directly or inserting an explicit swap step to convert ETH to the native parachain token. Either approach is conceptually straightforward.

E.7 | Resale Royalties

The current `transfer_ownership` extrinsic has no creator share on secondary-market transfers. CCRMS could add an optional per-content `resale_royalty_basis_points`

field; the implementation challenge is verifying the declared sale price, which suggests resale royalties would work best in conjunction with an on-chain marketplace pallet that records sale prices as part of the transfer transaction.

E.8 | Capability-Based Delegation for Cross-Chain Transfers

The conservative Finding B remediation rules out delegated transfers. A future iteration could relax this with a capability-based authorization model (the owner pre-authorizes a specific delegate for a specific operation and time window), which is well-explored territory in capability-based security. The implementation challenge is the storage layout for capability records and the revocation lifecycle.

E.9 | Monte Carlo Simulation Extensions

Future extensions could incorporate creator psychology (risk aversion, platform-switching costs, brand attachment), model network-effect dynamics at critical creator mass, add longitudinal modeling over multiple years, and model regulatory effects from the EU AI Act or MiCA enforcement.

E.10 | Formal Verification

Following Maric et al. (2025) on Isabelle/HOL verification of cross-chain bridges, similar techniques could be applied to the CCRMS authorization model, the royalty distribution algorithm, and the cross-chain operation pattern. Formal verification is expensive but offers guarantees that empirical testing cannot match, and is worthwhile for any production deployment handling significant value.

E.11 | JAM Transition

JAM (the Join-Accumulate Machine) represents Polkadot's architectural successor to the existing relay chain. The testnet was launched in January 2026 with the participation of 43 implementation teams; the mainnet is scheduled for deployment in mid-to-late 2026. CCRMS does not incorporate any JAM-specific features, and our implementation

specifications provide sufficiently detailed architectural decisions to facilitate potential future migration without necessitating a complete rebuild.

Work Plan and Timeline

We structured the work into six phases, originally planned across 24 weeks:

- **Phase 1 (Weeks 1–6): Research and Literature Review.** Establishes the academic and commercial context. Produces the initial 78-source literature review (later expanded to 100).
- **Phase 2 (Weeks 7–10): Design Conceptualization.** Translates high-level concepts into the system design and methodology.
- **Phase 3 (Weeks 11–12): Architecture Finalization.** Bridges the conceptual design to a concrete implementation plan.
- **Phase 4 (Weeks 13–16): Core Implementation.** The primary build phase: pallet-content-rights, runtime configuration, contract layer, XCM, Snowbridge integration.
- **Phase 5 (Weeks 17–20): Testing and Evaluation.** Unit tests, benchmarks, security audit, reliability test, comparative benchmark, Monte Carlo simulation, and literature review version 2.
- **Phase 6 (Weeks 21–24): Finalization.** Dissertation writing, implementation specifications, and LaTeX submission preparation.

References to “Phase N” throughout the dissertation denote work produced during that phase.

References

- 360iResearch. Blockchain for Content Monetization Market Size 2025-2030. Technical report, 360iResearch, 2025. URL <https://www.360iresearch.com/library/intelligence/blockchain-for-content-monetization>. Market Research Report.
- H. Abbas et al. Analysis of Polkadot: Architecture, Internals, and Contradictions. In *IEEE Blockchain 2022*, 2022. URL https://www.cri-lab.net/wp-content/uploads/2022/07/IEEE_Blockchain_2022_Polkadot_Architecture_Limitations.pdf.
- S. S. M. Abdul et al. CrossDeFi: A Novel Cross-Chain Communication Protocol. *Future Internet*, 16:314, 2024.
- S. J. Alotaibi. Using smart contracts in the proposed blockchain framework for an identity management system based on the Internet of Things. *International Journal of Distributed Systems Technology*, 14(1):1–22, 2023.
- L. Ante. Smart contracts on the blockchain – A bibliometric analysis and review. *SSRN Electronic Journal*, 2020. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3576393.
- Apple Inc. Apple Music Insights Royalty Rate. Apple for Artists Support, 2025. URL <https://artists.apple.com/support/1124-apple-music-insights-royalty-rate>. Accessed May 2026. Apple Music pays approximately 52% to labels; platform retains approximately 48%.
- M. Atzori. Blockchain Technology and Decentralized Governance: Is the State Still Necessary? *Journal of Governance and Regulation*, 6(1):45–62, 2017. doi: 10.22495/jgr_v6_i1_p5.
- A. P. Balcerzak et al. Blockchain technology and smart contracts in decentralized governance systems. *Administrative Sciences*, 12(3):96, 2022. doi: 10.3390/admsci12030096.
- A. Baliga. Understanding Blockchain Consensus Models. Technical report, Persistent Systems, 2020. URL <https://www.persistent.com/wp-content/uploads/2017/04/WP-Understanding-Blockchain-Consensus-Models.pdf>.
- P. Banerjee et al. Reliable, Fair and Decentralized Marketplace for Content Sharing Using Blockchain. arXiv:2009.11033, 2020. URL <https://arxiv.org/pdf/2009.11033>.

- A. Bigiotti, L. Mostarda, A. Navarra, A. Pinna, D. Sestili, R. Tonelli, and M. Vaccargiu. Blockchain Agnostic Protocols: An Analysis of the State of the Art. *ACM Computing Surveys*, 58(2):1–36, 2025. doi: 10.1145/3758089.
- K. Busch. Web3: A Proposed Blockchain-Based, Decentralized Web. Technical Report IF12075, Congressional Research Service, 2022. URL https://www.congress.gov/crs_external_products/IF/PDF/IF12075/IF12075.2.pdf.
- V. Buterin. Long term L1 execution layer proposal: replace the EVM with RISC-V. Ethereum Magicians Forum, 2025. URL <https://www.coindesk.com/tech/2025/04/21/vitalik-buterin-proposes-replacing-ethereums-evm-with-risc-v>. Posted April 20, 2025; coverage via CoinDesk.
- I. Buzu. Blockchain, smart contracts and copyright management disruption. SSRN, 2021. URL https://papers.ssrn.com/sol3/Delivery.cfm/SSRN_ID3759260_code3591422.pdf.
- M. A. Cabot-Nadal, M. Payeras-Capellà, M. Mut-Puigserver, and A. Soto-Fernández. Improving the Token ERC-721 Implementation for Selective Receipt: Rejectable NFTs. In *2022 6th International Conference on System Reliability and Safety (ICSRS)*, pages 243–250. IEEE, 2022. doi: 10.1109/ICSRS56243.2022.10067494.
- Y. Cao, J. Cao, D. Bai, L. Wen, Y. Liu, and R. Li. MAP the Blockchain World: A Trustless and Scalable Blockchain Interoperability Protocol for Cross-chain Applications. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*. ACM, 2025. doi: 10.1145/3696410.3714867.
- Castweet Team. Castweet – A Multi-broadcast Platform Based on Blockchain Incentives for Everyone. Technical report, Castweet, 2019. URL <https://resources.cryptocompare.com/asset-management/17402/1728919640132.pdf>. Whitepaper.
- M. Chegenizadeh, S. Rafati Niya, and C. J. Tessone. Multi-agent Privacy-Preserving Micro-payment Verification for Offline Resource-Constrained Devices Using Smart Contracts. In *Mathematical Research for Blockchain Economy (MARBLE 2024)*, Lecture Notes in Operations Research. Springer, 2024. doi: 10.1007/978-3-031-68974-1_7.
- F. Chen et al. Blockchain for Internet of Things applications: A review and open issues. *Journal of Network and Computer Applications*, 173:102854, 2020. doi: 10.1016/j.jnca.2020.102854.
- K. Chen et al. A Trusted Reputation Management Scheme for Cross-Chain Transactions. *Sensors*, 23:6033, 2023.
- M. Cherdakov, A. Kudashkin, Y. Madhwal, and Y. Yanovich. Web3 Based Digital Rights Management in the Music Industry. In *Proceedings of the 2023 6th International Conference on Blockchain Technology and Applications (ICBTA)*. ACM, 2023. doi: 10.1145/3651655.3651667.
- R. F. Ciriello, A. C. G. Torbensen, M. R. P. Hansen, and C. Müller-Bloch. Blockchain-based digital rights management systems: Design principles for the music industry. *Electronic Markets*, 33(1), 2023. doi: 10.1007/s12525-023-00628-5.

- Coinbase and x402 Foundation. x402 Protocol: HTTP-native blockchain micropayments. CoinPaprika, 2025. URL <https://coinpaprika.com/education/x402-ecosystem-instant-blockchain-payments-for-apis-and-ai-agents/>.
- CommuniPass. Creator Monetization in 2026: The 5 Models That Actually Generate Recurring Revenue. CommuniPass Blog, 2026. URL <https://communipass.com/blog/creator-monetization-in-2026-the-5-models-that-actually-generate-recurring-revenues/>.
- T. Cutts. Smart Contracts and Consumers. *West Virginia Law Review*, 2019.
- P. Darshan, J. S. Rohan, R. Rajesh, M. Ruchitha, S. Kamath, and M. N. Manas. Intellectual Property Rights and Entrepreneurship in the NFT Ecosystem: Legal Frameworks, Business Models, and Innovation Opportunities. arXiv:2507.00172, 2025. URL <https://arxiv.org/html/2507.00172v1>.
- C. Davik. The Art Of NFTs: Copyright, Contracts, And The Fallacy Of Ownership. *Georgia Law Review*, 2025.
- Z. Deng, C. Tang, T. Li, P. Abla, Q. Chen, W. Liang, and D. He. Enhancing Blockchain Cross-Chain Interoperability: A Comprehensive Survey. arXiv:2505.04934, 2025. URL <https://arxiv.org/abs/2505.04934>.
- K. S. Divyashree and A. Mishra. Blockchain technology in financial sector and its legal implications. In *Lecture Notes in Networks and Systems*, volume 478, pages 219–231. 2023.
- I. Dondjio and A. Kazamias. A Blockchain Framework for Digital Asset Ownership and Transfer in Succession. In *20th European, Mediterranean, and Middle Eastern Conference on Information Systems (EMCIS 2023)*, volume 501 of *Lecture Notes in Business Information Processing*. Springer, 2024. doi: 10.1007/978-3-031-56478-9_7.
- A. Dreyer et al. Can I mint an NFT with that?: Avoiding right of publicity and trademark litigation risks in the brave new world of NFTs. Skadden Publications, 2021. URL <https://www.skadden.com/-/media/files/publications/2021/05/canimintannftwiththatavoidingrightofpublicityandtr.pdf>.
- Eurojust. Non-Fungible Tokens and Intellectual Property Rights – Can the use of NFTs lead to IP infringements? Technical report, European Union Agency for Criminal Justice Cooperation, 2024. URL <https://www.eurojust.europa.eu/sites/default/files/assets/eurojust-nfts-intellectual-property-rights-flyer.pdf>.
- European Parliament. Report on copyright and generative artificial intelligence – opportunities and challenges. Technical Report A10-0019/2026, European Parliament, 2026. URL https://www.europarl.europa.eu/doceo/document/A-10-2026-0019_EN.html.
- European Securities and Markets Authority. MiCA Regulation: Markets in Crypto-Assets. Technical report, ESMA, 2025. URL <https://www.esma.europa.eu/esmas-activities/digital-finance-and-innovation/markets-crypto-assets-regulation-mica>. Full enforcement by July 2026.
- T. Feng et al. Smart contract model for complex reality transaction. 2019.

- M. Finck. *Blockchain regulation and governance in Europe*. Cambridge University Press, 2018. URL https://papers.ssrn.com/sol3/Delivery.cfm/SSRN_ID3351542_code3242224.pdf. Excerpt available via SSRN.
- A. Garba et al. A digital rights management system based on a scalable blockchain. Copenhagen Business School Research, 2021. URL https://research-api.cbs.dk/ws/portalfiles/portal/93314574/abba_garba_et_al_a_digital_rights_management_system_based_on_a_scalable_blockchain_publishersversion.pdf.
- J. Gatto et al. NFT License Breakdown: Exploring Different Marketplaces and Associated License Issues. Law of the Ledger, 2021. URL <https://www.lawofthelledger.com/wp-content/uploads/sites/15/2021/09/NFT-License-Breakdown-Article-0921.pdf>.
- D. Gogel et al. DeFi Beyond the Hype: The Emerging World of Decentralized Finance. Technical report, Wharton Blockchain and Digital Asset Project, 2021. URL <https://wifpr.wharton.upenn.edu/wp-content/uploads/2021/05/DeFi-Beyond-the-Hype.pdf>.
- A. Gomaa. A DRM Solution for Online Content Using Blockchain – A Music Perspective. In *INFOCOMP 2019*, 2019. URL https://personales.upv.es/thinkmind/dl/conferences/infocomp/infocomp_2019/infocomp_2019_1_30_60048.pdf.
- Google LLC. YouTube Partner Program: How ad revenue is shared. YouTube Help Center, 2025. URL <https://support.google.com/youtube/answer/72902>. Accessed May 2026. YouTube retains 45% of ad revenue; creators receive 55%.
- P. Goyal et al. Secure Incentivization for Decentralized Content Delivery. In *USENIX Workshop on Hot Topics in Edge Computing (HotEdge 2019)*, 2019. URL https://www.usenix.org/system/files/hotedge19-paper-goyal_0.pdf.
- Grand View Research. Digital Rights Management Market Size, Share & Trends Analysis Report. Technical report, Grand View Research, 2025. URL <https://www.grandviewresearch.com/industry-analysis/digital-rights-management-market>.
- A. Guadamuz. The EU’s Artificial Intelligence Act and copyright. *The Journal of World Intellectual Property*, 28:213–219, 2025. doi: 10.1111/jwip.12330.
- R. Habermeier et al. Polkadot’s messaging scheme. Web3 Foundation Research, Medium, 2021. URL <https://medium.com/web3foundation/polkadots-messaging-scheme-b1ec560908b7>.
- B. Hanneke, O. Hinz, J. Pfeiffer, and W. M. P. van der Aalst. The Internet of Value: Unleashing the Blockchain’s Potential with Tokenization. *Business & Information Systems Engineering*, 66(4):411–419, 2024. doi: 10.1007/s12599-024-00883-6.
- A. R. Hevner, S. T. March, J. Park, and S. Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.
- Information (MDPI). Library systems and digital-rights management: Towards a blockchain-based solution for enhanced privacy and security. *Information*, 17(2):137, 2026.

- ink! Alliance. Discontinuation of ink: Rust smart contract language. Polkadot Forum, 2026. URL <https://forum.polkadot.network/t/discontinuation-of-ink-rust-smart-contract-language/16849>. Posted January 27, 2026.
- K. Khan. Decentralized Video Streaming: Unleashing the Potential through Blockchain-Powered Platforms. *International Journal of Modern Research in Applied Physics (IJMRAP)*, 6(7), 2024.
- Kraken Intelligence. Polkadot & Kusama Parachains Primer. Technical report, Kraken, 2021. URL <https://cognizium.io/uploads/resources/Kraken%20Intelligence%20-%20Polkadot%20and%20Kusama%20Parachains%20Primer%20-%202021%20Sep.pdf>.
- S. Kulkarani et al. Blockchain Enabled Smart Contracts for Digital Assets. *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, 2024.
- R. K. Kumar et al. Revolutionizing digital ownership: Examining the perks of a Polkadot-based NFT marketplace. Technical report, ResearchGate, 2023.
- V. Kumar, I. Budhiraja, A. Jabbari, et al. Efficient blockchain interoperability design for cross-chain transactions in future internet-of-value. *Peer-to-Peer Networking and Applications*, 18:110, 2025. doi: 10.1007/s12083-025-01941-w.
- S. Lahiri et al. Multidisciplinary perspectives on Non-Fungible Tokens (NFTs): a comprehensive meta-analysis. *Digital Economy and Sustainable Development*, 4:3, 2026. doi: 10.1007/s44265-026-00078-9.
- L. Li et al. Why Blockchain-Based Digital Assets Are Owned on Decentralized Metaverse Platforms? In *Hawaii International Conference on System Sciences (HICSS)*, 2024. URL <https://scholarspace.manoa.hawaii.edu/server/api/core/bitstreams/da7d3ba9-379e-48a8-98e8-18b2bc293b1f/content>.
- Q. Li, X. Wen, and Z. Wu. Evaluating blockchain adoption barriers in China's IP trade and protection internationally. *Frontiers in Blockchain*, 8, 2025a.
- W. Li, Z. Liu, J. Chen, Z. Liu, and Q. He. Towards blockchain interoperability: a comprehensive survey on cross-chain solutions. *Blockchain: Research and Applications*, 6(3):100286, 2025b.
- S. Y. Lin et al. A survey of application research based on blockchain smart contract. *Wireless Networks*, 28: 635–690, 2022. doi: 10.1007/s11276-021-02874-2.
- S. L. Madapati and N. R. Pradhan. Decentralizing video copyright protection: a novel blockchain-enabled framework with performance evaluation. *Frontiers in Artificial Intelligence*, 8, 2025.
- H. Mao et al. A Survey on Cross-Chain Technology: Challenges, Development, and Prospect. 2022.
- D. Mareckova. *Blockchain and Collective Rights Management of Copyright and Related Rights at the Global Level: The Case of the Music Industry*. PhD thesis, OAPEN, 2024. URL https://library.oapen.org/bitstream/handle/20.500.12657/94575/oa_pdf-057-1731486746.pdf.
- F. Maric, B. Scholz, and P. Subotic. Formal Verification of a Fail-Safe Cross-Chain Bridge. In *6th International Workshop on Formal Methods for Blockchains (FMBC 2025)*, volume 129 of *OASICS*, 2025. URL <https://drops.dagstuhl.de/entities/document/10.4230/OASICS.FMBC.2025.8>.

- MarketsandMarkets. Digital Rights Management Market by Application. Technical report, MarketsandMarkets Research, July 2025. URL <https://www.marketsandmarkets.com/Market-Reports/digital-rights-management-market-152806525.html>.
- N. Mauro. Smart contracts for decentralized business models in the electricity market: a consumer protection perspective. Master's thesis, European Master in Law and Economics (EMLE), 2022. URL https://emle.org/wp-content/uploads/2023/01/EMLE_thesis_Mauro_Noemi.pdf.
- J. C. Mendoza-Tello, J. J. Sánchez-Lucas, and J. G. Orosco-Pantoja. A Model to Support the Music Streaming Service Using Blockchain-Based Smart Contracts. In *Applied Technologies (ICAT 2023)*, volume 2051 of *Communications in Computer and Information Science*. Springer, 2024. doi: 10.1007/978-3-031-58950-8_10.
- R. Moncada et al. DLT-enabled Platforms: Distribution and Management of Media Content. Technical report, Mediaverse Project, 2021. URL https://mediaverse-project.eu/wp-content/uploads/2021/09/Research-Note-Blockchain-based-Platforms_integrated.pdf.
- Mordor Intelligence. Digital Rights Management (DRM) Market – Size, Share & Industry Analysis. Technical report, Mordor Intelligence, January 2026. URL <https://www.mordorintelligence.com/industry-reports/digital-rights-management-drm-market>.
- M. F. Moreno et al. A Knowledge-Oriented Approach to Enhance Integration and Communicability in the Polkadot Ecosystem. arXiv:2308.00735, 2023. URL <https://arxiv.org/pdf/2308.00735>.
- D. Morháč, V. Valaštin, K. Košťál, and I. Kotuliak. ParaSpell XCM SDK: A New Protocol for Interoperability in Polkadot Paraverse. In *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*, pages 569–576. IEEE, 2023a. doi: 10.1109/BCCA58897.2023.10338906.
- D. Morháč, V. Valaštin, K. Košťál, and I. Kotuliak. Enhancing XCM Interoperability Across Polkadot Paraverse. In *2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–3. IEEE, 2023b. doi: 10.1109/ICBC56567.2023.10174872.
- D. Morháč, K. Košťál, V. Valaštin, et al. Unispell: universal adapter for interoperability in Polkadot Paraverse. *Cluster Computing*, 28:311, 2025. doi: 10.1007/s10586-025-05183-6.
- M. Munson. A Blockchain Controlled Content Fabric. Video Services Forum, 2019. URL https://static.vsf.tv/Meetings/2019/2019_DC/Munson_BlockchainFabric.pdf.
- M. Munson et al. Blockchain Content Fabric for Multi-Chain Content Ownership and Distribution. Technical report, Eluvio, 2022. URL <https://eluv.io/3e39198441b1d578bc0f.pdf>.
- M. Murray. NFT Ownership And Copyrights. *Indiana Law Review*, 56:367, 2022.
- Music Business Research. The Music Streaming Economy Part 14: The Artists' Share of the Music Streaming Pie. Music Business Research Blog, 2024. URL <https://musicbusinessresearch.wordpress.com/2024/09/16/the-music-streaming-economy-part-14-the-artists-share-of-the-music-streaming-pie/>. Accessed May 2026.
- P. Orero et al. The Visible Subtler: Blockchain Technology for Right Management and Minting. *Open Research Europe*, 3:26, 2023.

- O. Osemwegie. Decentralized Media Distribution: Exploring blockchain's role in copyright protection, monetization, and content ownership rights. *International Journal of Mechanical Technology and Marine Engineering*, 2025.
- Parity Technologies. Polkadot Parachain Slots. Polkadot Blog, 2021. URL <https://polkadot.com/blog/polkadot-parachain-slots/>. Describes Polkadot's support for up to 100 parachain slots and additive parallel throughput.
- Parity Technologies. ink! 3.0: Parity's Rust-based language for WASM smart contracts gets a major update. Parity Blog, 2022a. URL <https://www.parity.io/blog/ink-3-0-paritys-rust-based-language-gets-a-major-update>.
- Parity Technologies. What is Parity's ink!? Parity Blog, 2022b. URL <https://www.parity.io/blog/what-is-paritys-ink>.
- Parity Technologies. Polkadot Upgrade 2025: What You Need to Know. Parity Blog, 2025. URL <https://www.parity.io/blog/polkadot-upgrade-2025-what-you-need-to-know>.
- S. S. Patil et al. Blockchain Based Multimedia Content Protection Using Encryption in DRM (Digital Rights Management). *International Journal of Innovative Research in Technology (IJIRT)*, 2022.
- Polkadot Network. Polkadot Roundup 2025. Medium, 2025. URL <https://medium.com/polkadot-network/polkadot-roundup-2025-3c3c71c7e9c4>.
- L. Quan et al. Research and application of digital collection smart contract method based on Blockchain. 2024.
- R0GUE. ink! speaks Solidity on PolkaVM. Medium, 2025. URL <https://r0gue.medium.com/ink-solidity-abi-on-polkavm-c675c854efd3>. Posted August 2025.
- I. S. Rao, M. L. M. Kiah, M. M. Hameed, and Z. A. Memon. Scalability of blockchain: a comprehensive review and future research direction. *Cluster Computing*, 27:5547–5570, 2024. doi: 10.1007/s10586-023-04257-7.
- E. Reggianini. Standardizing On-chain IP Rights Management. Technical report, BlockStand, 2025. URL https://blockstand.eu/blockstand/uploads/2025/05/Standardizing-On-chain-IP-Rights-Management_Reggianini.pdf.
- K. Ren, N.-M. Ho, D. Loghin, T.-T. Nguyen, B. C. Ooi, Q.-T. Ta, and F. Zhu. Interoperability in Blockchain: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12750–12769, 2023. doi: 10.1109/TKDE.2023.3275220.
- B. Salman et al. NFT-Based Secure Platform for Copyright Images (NFT-SPCI). *Technix International Journal for Engineering Research (TIJER)*, 2023.
- H. O. Sevim. A Survey on Trustless Cross-chain Interoperability Solutions in On-chain Finance. In *Distributed Ledger Technology Workshop (DLT 2024)*, 2024. URL https://dlt2024.di.unito.it/wp-content/uploads/2024/05/DLT2024_paper_14.pdf.

- A. Shah. Automating Royalties: A Framework of Smart Contracts for Cross-Platform Content Revenue. *International Journal for Multidisciplinary Research (IJFMR)*, 2025.
- L. Shumylyak et al. Practical Implementation of Smart Contracts for Payment of Digital Goods. In *CEUR Workshop Proceedings*, volume 3373, 2023. URL <https://ceur-ws.org/Vol-3373/short1.pdf>.
- R. Singh et al. Insights into research on blockchain for smart contracts: A bibliometric analysis. *Multimedia Tools and Applications*, 83:1–25, 2024a. doi: 10.1007/s11042-023-15647-1.
- R. Singh et al. Blockchain-Based Digital Rights Management and Its Impact on English Language Resources in Libraries. IGI Global, 2024b. URL <https://www.igi-global.com/viewtitle.aspx?TitleId=360345&isxn=9798369396162>.
- Smart Contracts Alliance. Smart Contracts: 12 Use Cases for Business & Beyond. Technical report, Chamber of Digital Commerce, 2016. URL https://d3h0qzni6h08fz.cloudfront.net/Smart-Contracts-12-Use-Cases-for-Business-and-Beyond_Chamber-of-Digital-Commerce.pdf.
- Smart Contracts Alliance. Smart Contracts: Is the Law Ready? Technical report, Chamber of Digital Commerce, 2018. URL <https://lowellmilkeninstitute.law.ucla.edu/wp-content/uploads/2018/08/Smart-Contracts-Whitepaper.pdf>.
- Snowfork. Snowbridge V2 Is Live! Polkadot Forum, 2025a. URL <https://forum.polkadot.network/t/snowbridge-v2-is-live/15844>. Posted November 7, 2025.
- Snowfork. Snowbridge 2025/2026 Stabilized. Polkasembly Proposal #3313, 2025b. URL <https://polkadot.polkasembly.io/post/3313>.
- Spotify AB. Understanding Spotify Royalties. Spotify for Artists Support, 2025. URL <https://support.spotify.com/us/artists/article/understanding-spotify-royalties/>. Accessed May 2026. Spotify pays approximately 70% of revenue to rights holders.
- Steem Team. Steem, an incentivized, blockchain-based, public content platform. Technical report, Steem, 2025. URL <https://steem.com/SteemWhitePaper.pdf>. Whitepaper.
- Story Foundation. Story Mainnet Launch. CoinDesk, 2025. URL <https://www.coindesk.com/tech/2025/02/13/story-protocol-launches-to-let-people-to-register-ip-and-get-paid-for-it/>. Posted February 13, 2025.
- R. Tao. The Study of Blockchain Technology for Copyright Protection and Resource Sharing Models in Library Digital Resources. 2025.
- TokenInsight Research. 2021 Polkadot Ecosystem Research Report. Technical report, TokenInsight, 2021. URL <https://downloads.coindesk.com/research/2021-Polkadot-Ecosystem-Research-Report.pdf>.
- K. Upadhyay et al. Paradigm Shift from Paper Contracts to Smart Contracts. 2021.
- USPTO. Non-Fungible Tokens and Intellectual Property: A report to Congress. Technical report, U.S. Patent and Trademark Office, 2024. URL <https://www.copyright.gov/policy/nft-study/Joint-USPTO-USCO-Report-on-NFTs-and-Intellectual-Property.pdf>.

- V. Valaštin et al. Protocol for Unifying Cross-Chain Liquidity on Polkadot. *Frontiers in Blockchain*, 2024. doi: 10.3389/fbloc.2024.1413840.
- P. E. Velmovitsky et al. Blockchain applications in health care and public health: Increased transparency. *JMIR Medical Informatics*, 9(6):e25893, 2021. doi: 10.2196/25893.
- D. Virovets et al. A Framework for Decentralized Payment Instrument Integration with Artificial Intelligence, Big Data, and Digital Identities. In *CEUR Workshop Proceedings*, volume 3991, 2025. URL <https://ceur-ws.org/Vol-3991/paper33.pdf>.
- M. Wamugo. Content monetization: non-fungible tokens as a new revenue stream for the media sector in Kenya. Master's thesis, Aga Khan University, 2024. URL https://ecommons.aku.edu/cgi/viewcontent.cgi?article=3232&context=theses_dissertations.
- Z. Wang, W. Feng, M. Huang, S. Feng, S. Mo, and Y. Li. Blockchain-based information security protection mechanism for the traceability of intellectual property transactions. *Sensors*, 25(10):3064, 2025.
- G. Wood. Polkadot: Vision for a heterogeneous multi-chain framework. Technical report, Web3 Foundation, 2016. URL <https://polkadot.network/whitepaper>. White Paper.
- G. Wood. XCM: The cross-consensus message format. Polkadot Blog, 2022. URL <https://www.polkadot.network/blog/xcm-the-cross-consensus-message-format>.
- G. Wood. JAM: The Join-Accumulate Machine (Gray Paper). Technical report, Web3 Foundation, 2024. URL <https://wiki.polkadot.com/learn/learn-jam-chain/>. Version 0.7.1+, updated through 2025.
- R. Xie and Y. Liu. Research on copyright protection of digital works based on multi-blockchain. *Recent Advances in Computer Science and Communications*, 15(8):1126–1134, 2022. doi: 10.2174/2666255815666220204092947.
- R. Xie and M. Tang. A digital resource copyright protection scheme based on blockchain cross-chain technology. *Heliyon*, 10(16):e34781, 2024. doi: 10.1016/j.heliyon.2024.e34781.
- X. Yi, Y. Zhou, Y. Lin, et al. Digital rights management scheme based on redactable blockchain and perceptual hash. *Peer-to-Peer Networking and Applications*, 16:2630–2648, 2023. doi: 10.1007/s12083-023-01552-3.
- Z. Yin et al. Bool Network: An Open, Distributed, Secure Cross-chain Notary Platform. IACR ePrint 2022/1290, 2022. URL <https://eprint.iacr.org/2022/1290.pdf>.
- M. Zeggari et al. An Efficient and Decentralized Blockchain-based Commercial Alternative. IACR ePrint 2022/1440, 2022. URL <https://eprint.iacr.org/2022/1440.pdf>.
- H. Zheng et al. Decentagram: Highly-Available Decentralized Publish/Subscribe Systems. 2024.
- Z. Zheng et al. An Overview on Smart Contracts: Challenges, Advances and Platforms. arXiv:1912.10370, 2019. URL <https://arxiv.org/pdf/1912.10370>.
- Q. Zhou et al. A Study on Blockchain Interoperability Mechanism. *Journal of System and Management Sciences*, 11(4), 2021.